Make LIDAR light.

SLIM: Scalable and Lightweight LiDAR Mapping in Urban Environments

Zehuan Yu, Zhijian Qiao, Wenyi Liu, Huan Yin, *Member, IEEE*, and Shaojie Shen

***Abstract*—Light detection and ranging (LiDAR) point cloud maps are extensively utilized on roads for robot navigation due to their high consistency. However, dense point clouds face challenges of high memory consumption and reduced maintainability for long-term operations. In this study, we introduce scalable and lightweight LiDAR mapping (SLIM), a scalable and lightweight mapping system for long-term LiDAR mapping in urban environments. The system begins by parameterizing structural point clouds into lines and planes. These lightweight and structural representations meet the requirements of map merging, pose graph optimization, and bundle adjustment, ensuring incremental management and local consistency. For long-term operations, a map-centric nonlinear factor recovery method is designed to sparsify poses while preserving mapping accuracy. We validate the SLIM system with multisession real-world LiDAR data from classical LiDAR mapping datasets, including KITTI, NCLT, HeLiPR, and M2DGR. The experiments demonstrate its capabilities in mapping accuracy, lightweightness, and scalability. Map reuse is also verified through map-based robot localization. Finally, with multisession LiDAR data, the SLIM system provides a globally consistent map with low memory consumption (∼ 130 KB/km on KITTI).**

***Index Terms*—Light detection and ranging (LiDAR), long-term simultaneous localization and mapping (SLAM), mobile robot, nonlinear factor recovery.**

I. Introduction

MAPPING is a fundamental capability for mobile robots navigating in various environments. Modern techniques, such as simultaneous localization and mapping (SLAM), provide metric maps that facilitate subsequent tasks like online map-based localization and planning. Over the past decade, light detection and ranging (LiDAR) has emerged as the gold standard for robotic mapping. Significant advancements in LiDAR-based mapping have been validated in diverse environments, such as underground areas [1] and forests [2].

The urban environment is the primary setting of our daily lives. Mapping in urban environments has a rich history, dating back to the development of autonomous vehicles during the DARPA Urban Challenge [3]. With advancements in LiDAR SLAM, mobile robots are now well developed to handle mapping tasks in well-structured urban areas. However, deploying existing systems on mobile robots for long-term use presents several challenges in urban environments, stated as follows.

1) LiDAR sensors generate 3-D point clouds directly. Conventional mapping systems have achieved accurate robot pose estimation and LiDAR mapping using dense representations like points [4], voxels [5], and meshes [6]. While these methods enable precise environmental modeling, they are not memory-efficient for practical applications, particularly on resource-constrained platforms operating in large-scale urban environments.
2) High-level or implicit representations, such as objects or neural functions, offer memory efficiency but may not ensure sufficient accuracy for precise localization in urban environments. Reusing the generated map is crucial for robotic applications, making the integration of low-level geometric information essential in LiDAR mapping systems.
3) A limitation of many existing LiDAR mapping systems is their design for single-session tests, lacking consideration for long-term deployment. Scalability becomes crucial for multisession LiDAR mapping systems in two aspects: globally merging multiple map sessions and maintaining the consistency and size of local maps as the number of sessions increases.

Generally, urban environments are characterized by structured and manually deployed settings, such as poles and traffic signs, which provide strong prior knowledge for long-term mapping. Similar structures are found in indoor environments. Kimera by Rosinol et al. [7] modeled the semantic scene graph in such settings. Inspired by these studies, we designed a *scalable* and *lightweight* LiDAR mapping (SLIM) system that leverages the structuralism of urban environments. Fig. 1 presents the maps generated from SLIM, compared to widely used point cloud maps. Specifically, the key contributions span from the front-end representation to the back-end smoothing, as follows.

1) We propose to parameterize lines and planes as memory-efficient representations, which encode geometric information and are also suitable for map merging.
2) Subsequently, pose graph optimization (PGO) and bundle adjustment (BA) are designed to refine the LiDAR mapping in a coarse-to-fine manner.

Received 26 January 2025; accepted 11 March 2025. Date of publication 24 March 2025; date of current version 10 April 2025. This work was supported in part by HKUST Postgraduate Studentship and in part by HKUST-DJI Joint Innovation Laboratory. This article was recommended for publication by Associate Editor Andreas Nuechter and Editor Javier Civera upon evaluation of the reviewers' comments. *(Corresponding author: Huan Yin.)*

Zehuan Yu, Zhijian Qiao, Huan Yin, and Shaojie Shen are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: zyuay@connect.ust.hk; zqiaoac@connect.ust.hk; eehyin@ust.hk; eeshaojie@ust.hk).

Wenyi Liu is with the Department of Mechanism Engineering, The University of Hong Kong, Hong Kong, China (e-mail: liuwenyi@connect.hku.hk).

This article has supplementary downloadable material available at https://doi.org/10.1109/TRO.2025.3554400, provided by the authors.

Digital Object Identifier 10.1109/TRO.2025.3554400

1941-0468 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

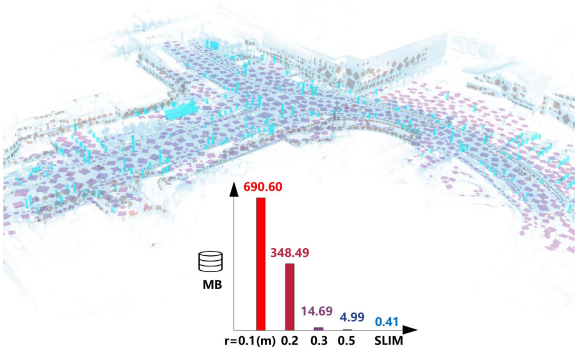


Fig. 1. Demonstration of SLIM using the HeLiPR dataset [9]. Best viewed zoomed in and in color. SLIM provides parameterized maps with lines (colored in cyan) and planes (colored in magenta). For comparison, we also present point clouds (in light blue), which are widely used in conventional LiDAR mapping systems. More mapping results are presented in Fig. 10. Furthermore, we compare the memory consumption of SLIM-generated maps with point cloud maps of varying densities (downsampled with radius r). In this HeLiPR session, SLIM maps are inherently more lightweight than LiDAR point cloud maps, with memory usage significantly reduced from 690.60 MB (point clouds) to just 0.41 MB (SLIM).

3) We introduce a map-centric marginalization scheme to manage map size as sessions increase, ensuring scalability for long-term maintenance. The key contribution is an efficient nonlinear factor recovery (NFR) method based on parameterized landmarks.
4) We validate the SLIM system on four different multisession datasets in terms of accuracy, lightweightness, and scalability.

The proposed SLIM system is an extended version of our previous work in [8]. This enhanced version introduces improvements across three key aspects. First, it no longer relies on semantic information for line and plane extraction, focusing solely on geometric characteristics. We believe that additional semantic information from visual perception can enhance the mapping performance in practice. Second, the map merge module has been redesigned to improve robustness and efficiency for urban mapping. Lastly, we introduce a map-centric nonlinear factor recovery method to improve the system's scalability. In summary, these improvements make the SLIM system more applicable for long-term urban mapping.

The rest of this article is organized as follows. Section II presents related work on LiDAR mapping approaches. Section III provides an overview of the designed SLIM system. Section IV details the representations, map merge, and optimization within the SLIM system. Section V introduces our proposed map-centric marginalization for long-term operation. We validate the proposed system in real-world scenarios in Section VI. Discussions and limitations are presented in Section VII for in-depth analysis. Finally, Section VIII concludes this article.

II. Related Work

This section first focuses on multisession LiDAR SLAM, a prominent research topic over the past five years. Subsequently, we discuss the representations of LiDAR SLAM and long-term mapping techniques.

A. Multisession LiDAR SLAM

Research on LiDAR SLAM has achieved significant success. Representative works include LOAM [10], Fast-LIO2 [4], DLO [11], and BALM2 [12]. These can be categorized into two types: the majority [4], [10], [11], [13], [14] adopt a scan-to-map scheme with a fixed map, demonstrating high efficiency and accuracy. The second branch is BA-based methods, which parameterize map points as planes [12], [15], [16], [17], occupancy fields [18], and implicit maps [19]. BA-based methods optimize the map along with the pose to achieve higher accuracy compared to scan-to-map approaches. As LiDAR SLAM has developed, researchers have shifted focus towards multisession LiDAR SLAM. This involves assembling individual maps and poses from multiple sessions to construct a more complete and accurate globally consistent map, under either a distributed or centralized scheme.

The general pipeline for assembling multisession maps in different coordinates involves two key steps: loop closure detection and false loop pruning [20]. These steps aim to establish correspondences and eliminate incorrect ones, respectively. Several methods employ different approaches for loop closure detection. Maplab [21] and DiSCo-SLAM [22] utilize the Scan Context descriptor [23] to detect loops between maps. However, handcrafted methods may struggle with significant viewpoint changes and have limited descriptor capabilities. In contrast, AutoMerge [24] uses a more powerful deep learning-based method, an improved version of FusionVLAD [25], to detect loops in large-scale environments. To prune false loops, various heuristics are introduced to reject potential false positives, including quality assessment of point cloud registration [26], loop closure prioritization [27], and ensuring sequential consistency [28]. While heuristic methods provide a rapid means of rejecting obvious false loops, they can still suffer from appearance ambiguities. Therefore, most multisession LiDAR SLAM approaches [22], [29], [30], [31] employ pairwise consistent measurement (PCM) set maximization [32] to prune outliers. This involves finding the maximum geometrically consistent loop set, thereby decreasing false loop associations with similar appearances.

To ensure the global consistency of the merged map, PGO is employed to align the trajectories from individual sessions and to reduce the accumulated drift caused by LiDAR odometry. Latif et al. [33] introduced a novel algorithm, realizing, reversing, and recovering (RRR), to identify and reject loops that violate odometry constraints. To improve efficiency, LTA-OM [34] eliminates the need for re-estimating graph parameters by evaluating the residual outcomes of key point factors and determining whether to recover the backup graph based on these residuals. However, these methods, which involve the incremental addition and rejection of loops, face efficiency limitations due to the requirement of multiple optimization rounds. To address this issue, contemporary multisession SLAM systems, such as Kimera-Multi [30] and LAMP 2.0 [35], incorporate all loop factors into the M-Estimator optimization to reject outliers. Subsequently, graduated nonconvexity (GNC) [36] is employed to achieve robust PGO. In contrast to the aforementioned

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

approaches that focus on global consistency using PGO, HBA [37] employs LiDAR BA to enhance map consistency directly and introduces a hierarchical structure to reduce computation cost.

B. Representations for LiDAR SLAM

Map representation is a front-end and crucial component of LiDAR SLAM. In this section, we categorize map representations into two types: explicit and implicit. Modeling the world with these representations is closely coupled with specific robotic tasks. For multisession LiDAR mapping in this study, it is essential to consider both map memory consumption and localizability.

1) Explicit Representation: The explicit representations can be further categorized into dense spatial and sparse landmark-based representations. Dense representations typically include dense points [38], voxels [39], mesh [6], surfels [40], and TSDF [41]. These aim to construct comprehensive and detailed descriptions of occupied spaces or surfaces. However, dense representations could be a huge burden for map transmission and storage in multisession SLAM, often containing redundant information, such as complete occupancy details, which are unnecessary for urban navigation. Conversely, sparse representations, such as lines [42], [43], [44], [45], planes [12], [17], [46], [47], quadrics [48], and segments [49], could be better choices to describe large-scale urban environments. Among these, line and plane representations leverage the inherent characteristics of urban scenes, balancing high localization accuracy with minimal memory requirements.

2) Implicit Representation: Implicit representations in radiance [50], distance [51], and occupancy [52] fields have significantly influenced LiDAR SLAM research. SHINE-Mapping [53] presents an incremental implicit mapping system incorporating online learning of sparse octree node features. LONER [54] adopts a hierarchical feature grid encoding inspired by Ins-NGP, building a real-time neural implicit LiDAR SLAM system enhanced with LiDAR odometry. NeRF-LOAM [55] introduces a dynamic voxel embedding generation approach designed for large-scale scenario. These implicit neural LiDAR SLAM methods exhibit impressive mapping accuracy. However, they either necessitate mesh map reconstruction for localization or depend directly on the implicit map, constraining their operational frequency for real-time use and long-term scalable mapping.

C. Long-Term Mapping

Long-term mapping presents challenges for visual-based frameworks due to illumination changes [56], [57]. Unlike visual sensing, LiDAR sensing remains consistent and illumination-invariant across day–night variations, making it well suited for long-term mapping. At the front end, managing newly generated maps from robots has been extensively studied. Typically, map merging involves place recognition and point cloud registration techniques [20]. For long-term management, compressing point cloud size can reduce computational demands for online applications, utilizing observation-based [58] and geometric-based sampling [59].

Most works in the community focus on back-end processing for long-term mapping, specifically on the SLAM problem. Early filter-based methods, such as sparse extended information filters, demonstrated that SLAM could be solved in constant time by exploiting sparse structures [60], [61]. Modern SLAM is typically formulated as an optimization problem [62] using pose graphs (only poses) or factor graphs (poses and landmarks) [63], [64]. In pose graph-based SLAM, sensing information on poses can compact the graph structure [65], [66]. Kretzschmar and Stachniss [66] designed an information-aided long-term 2-D LiDAR mapping scheme where laser measurements with minimal information are discarded, and pose nodes are marginalized accordingly. The researchers use a Chow-Liu tree to maintain the marginalized pose graph. The main challenge with these approaches is modeling information (uncertainty), especially for dense 3-D LiDAR scans. GLC [67] is developed to remove nodes from SLAM graphs, bounding computational complexity for multisession mapping in the same area. Recent work by Doherty et al. [68] proposed selecting measurements by maximizing the algebraic connectivity of pose graphs. Similar measurement selection techniques have been designed in previous studies [69], [70] for long-term graph pruning.

In the NFR by Mazuran et al. [71], [72], graph sparsification is formulated as a Kullback–Leibler divergence minimization problem, with a closed-form solution available. The mean and covariance of maintained variables are re-estimated, i.e., recovered. The prior condition for NFR is the existence of a node removal method. This minimization can also be approached in a descent form, as shown in [73]. Jiang and Shen [74] proposed a new NFR framework for long-term visual mapping, focusing on landmark-based factors. Relevant back-end techniques have been applied to several SLAM systems, such as cooperative SLAM for marine robots [75] and visual-inertial odometry [76]. Inspired by these studies, we adopt a map-centric NFR into our SLIM system for long-term map maintenance. Considering the sparsity of urban environments, the closed-form solution is approximated using sparse matrix operations, as detailed in Section V.

III. System Overview

The system pipeline is illustrated in Fig. 2. Given sequential input LiDAR scans, SLIM first converts raw LiDAR point clouds into line and plane representations based on geometric properties. This vectorization pipeline is detailed in Section IV-A. These parameterized representations are compact and manageable for subsequent tasks. Lines and planes are accumulated using LiDAR odometry or with additional onboard odometry, resulting in lightweight landmarks in maps. To merge multisession maps, SLIM utilizes a global registration approach without place recognition, detailed in Section IV-B. Though the maps are unified in the same coordinate, the drift from odometry still exists. In Section IV-C, SLIM achieves map refinement (smoothing) through a coarse-to-fine approach: PGO and BA, designed based on the parameterized lines and planes. With increased

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

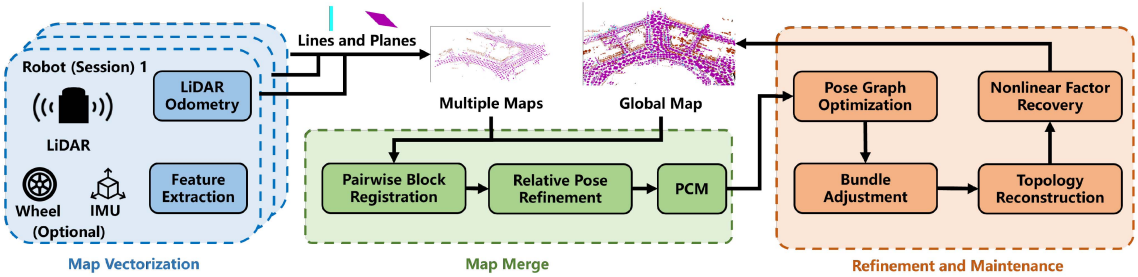


Fig. 2. System overview. The front-end map vectorization module extracts features and parameterizes planes to lanes. Then, the SLIM system generates one global lightweight map via map merge. The map refinement modules, including PGO and BA, smooth the mapping results only using lines and planes. The map-centric marginalization can bound the computational and storage requirements with increasing map sessions. Overall, SLIM is a centralized server that can achieve map vectorization, map merge and refinement for long-term LiDAR mapping.

Algorithm 1: SLIM Pipeline.

Input: Base Map $\mathcal{M}_{b,k-1}$ at $k-1$, New submap $\mathcal{M}_s$
Output: New Base Map $\mathcal{M}_{b,k}$ at k

$\{\mathcal{G}_{b_p}\}, \{\mathcal{G}_{s_q}\} = \texttt{partition}(\mathcal{M}_{b,k-1}, \mathcal{M}_s)$ IV-B1
$\{\mathbf{T}_{s_i}^{b_i}\} = \texttt{registerBlock}(\{\mathcal{G}_{b_p}\}, \{\mathcal{G}_{s_q}\})$ IV-B2
$\{\mathbf{T}_{s_i}^{b_i}\} = \texttt{filterLoop}(\{\mathbf{T}_{s_i}^{b_i}\})$ IV-B3
$\bar{\mathcal{M}}_{b,k} = \texttt{PGO}(\mathcal{M}_{b,k-1}, \mathcal{M}_s, \{\mathbf{T}_{s_i}^{b_i}\})$ IV-C1
$\bar{\mathcal{M}}_{b,k} = \texttt{BA}(\bar{\mathcal{M}}_{b,k})$ IV-C2
$\mathcal{M}_{b,k} = \texttt{Marginalization}(\bar{\mathcal{M}}_{b,k})$ V

merged map sessions, a map-centric nonlinear factor recovery is implemented to keep the number of poses manageable, as presented in Section V. All these operations mentioned above make the mapping pipeline scalable with increasing multisession LiDAR maps. We also summarize the pipeline in Algorithm 1. It is worth noting that the map landmarks consist solely of parameterized lines and planes after map vectorization, without storing any dense point clouds in the map.

IV. Map Representation, Merge and Refinement

We first present the front-end map vectorization in Section IV-A. Subsequently, the map merge function is introduced in Section IV-B to unify local maps within the global frame through place recognition and global registration. In Section IV-C, map refinement modules improve mapping accuracy using our proposed PGO and BA approaches.

A. Map Vectorization: From Point Clouds to Lines and Planes

1) Preprocessing and Feature Points Selection: Given sequential LiDAR scans from robots, we apply existing LiDAR odometry methods, such as LOAM [10] and KISS-ICP [14], to obtain local maps and sequential robot poses. Sensor fusion approaches [4] could also be an alternative. These methods provide dense point clouds and robot poses with drifts, serving as input for the SLIM system. Typically, robots or vehicles operate in different parts of urban environments, corresponding to different local frames. The SLIM system is designed to provide a consistent and lightweight map in a global frame.

The most common objects in urban environments are buildings, roads, and roadside poles, which can be categorized into two types of features: *lines* and *planes*. For line feature extraction, we initially divide points into their respective scan lines to restore the original scanning pattern. It is important that points on the same scan line are arranged in the order of scanned timestamp. Subsequently, we calculate the bilateral distance difference for a point on a scan line. Based on the distance difference values on both sides, we label points as negative gradient points, positive gradient points, and points with gradients on both sides with a threshold. We then identify pairs of neighboring negative and positive gradient points that are sufficiently close in distance. Extracting the points between them, as well as those with gradients on both sides, allows us to obtain a candidate point set. Finally, we employ clustering and line fitting methods to refine this candidate point set, finally obtaining multiple *line segments*. Planar points are selected based on the local structure of points. First, TRAVEL [77] is used to identify ground and nonground points. Hash octrees are then constructed separately for these two different types of points. Classical principal component analysis (PCA) is applied to identify planar points and convert them to *plane segments*. These line and plane segments are significantly fewer than the raw LiDAR points while preserving the original geometric information.

We define a *line observation* as $\mathbf{f}_{\mathcal{L}} := \langle \hat{\mathbf{p}}_a, \hat{\mathbf{p}}_b, \sqrt{\mathbf{\Lambda}_{\mathcal{L}}}, N \rangle$, and a *plane observation* as $\mathbf{f}_{\mathcal{S}} := \langle \hat{\mathbf{p}}_a, \hat{\mathbf{p}}_b, \hat{\mathbf{p}}_c, \sqrt{\mathbf{\Lambda}_{\mathcal{S}}}, N \rangle$. Two observation points are used to build residuals for line landmarks, while three are used for plane landmarks. To ensure that these selected sparse observation points effectively represent the local point cloud structure, their selection is based on the eigenvalue decomposition results of a local PCA. More specifically, we assume that all extracted LiDAR points of a single observation is $\{\mathbf{p}_i\}$ and their centroid is $\bar{\mathbf{p}}$. The computed eigenvalues are $\lambda_1 < \lambda_2 < \lambda_3$ and the corresponding eigenvectors are $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$. The two line observation points are defined as

$$\begin{aligned} \hat{\mathbf{p}}_a &= \bar{\mathbf{p}} + \sqrt{2\lambda_3}\mathbf{v}_3 \\ \hat{\mathbf{p}}_b &= \bar{\mathbf{p}} - \sqrt{2\lambda_3}\mathbf{v}_3. \end{aligned} \tag{1}$$

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

Similarly, the three plane observation points are defined as

$$\hat{\mathbf{p}}_a = \bar{\mathbf{p}} + \sqrt{2\lambda_2}\mathbf{v}_2$$
$$\hat{\mathbf{p}}_b = \bar{\mathbf{p}} - \sqrt{\frac{\lambda_2}{2}}\mathbf{v}_2 + \sqrt{\frac{\lambda_1}{2}}\mathbf{v}_1$$
$$\hat{\mathbf{p}}_c = \bar{\mathbf{p}} - \sqrt{\frac{\lambda_2}{2}}\mathbf{v}_2 - \sqrt{\frac{\lambda_1}{2}}\mathbf{v}_1. \quad (2)$$

It is worth noting that the degrees-of-freedom (DoF) for the line residuals of two observation points are 4, which is consistent with the DoF of line landmarks. Similarly, the DoF for the plane residuals of three observation points are 3, which is consistent with the DoF of plane landmarks. This design is optimal for our LiDAR BA and scalable mapping, as further elaborated in Sections IV and V.

We also introduce information matrices to describe uncertainties of extracted lines and planes. We define $\sqrt{\mathbf{\Lambda}} = \mathbf{I} \cdot (\sqrt{N/m}/\sigma)$ as the square root of the information matrix, where $(\sqrt{\mathbf{\Lambda}})^T\sqrt{\mathbf{\Lambda}} = \mathbf{\Lambda}$ and m is the number of observation points for a landmark. The term N represents the number of raw points in line or plane segments. The variance σ varies across different semantic categories. Specifically, σ is determined by the flatness of point cloud. We set $\sigma_r = 0.1, \sigma_b = 0.2, \sigma_p = 0.3$ for roads, buildings, and poles in the experimental validation.

2) Vectorized Mapping: Parameterizing the *observations* into *landmarks* is crucial for long-term maintenance. To reduce the complexity, we sample dense robot poses from odometry as *keyframes*. Regarding the landmarks, we define a *line landmark* as $\mathbf{l}_\mathcal{L} := \langle l_l, \mathbf{c}_\mathcal{L}, \mathbf{n}_\mathcal{L}, \mathbf{p}_\mathcal{L}, \{\mathbf{f}_{\mathcal{L}_i}\}\rangle$ and a *plane landmark* as $\mathbf{l}_\mathcal{S} := \langle l_s, \mathbf{c}_\mathcal{S}, \mathbf{n}_\mathcal{S}, \mathbf{p}_\mathcal{S}, \{\mathbf{f}_{\mathcal{S}_i}\}\rangle$, which include a label, centroid, normal, minimum parameter block, and their *observations* across different *keyframes*. Specifically, the normal is a directional vector for a line and a normal vector for a plane. The *observations* encodes that the landmark is associated with a specific *keyframes*, and this association can be obtained through centroid-based nearest neighbor searching when applying LiDAR odometry. Essentially, we have constructed a factor graph structure similar to *visual SLAM*.

The next step is to parameterize the landmarks to enable optimization modules for map refinement. Meanwhile, a minimum parameter block is needed to reduce the problem scale as mentioned. The point-normal form, i.e., the normal vectors $\mathbf{n}$ and random points $\mathbf{c}$, is overparameterized for lines and planes and unsuitable for optimization problems. To address this, we propose representing an infinite line as $\mathbf{p}_\mathcal{L} := \langle \alpha, \beta, x, y\rangle \in \mathbb{R}^4$, where α and β represent the direction, and x and y represent the offset translation on the xOy plane. Similarly, an infinite plane is formulated as $\mathbf{p}_\mathcal{S} := \langle \alpha, \beta, d\rangle \in \mathbb{R}^3$, where α and β represent the direction, and d represents the offset translation on the z-axis, thus formulating the desired minimum parameter blocks.

Fig. 3 illustrates how we obtain arbitrary lines and planes using our minimal parameters. Theoretically, all infinite lines and infinite planes could be transformed from the original line $\mathbf{u}_z$ and the original plane xOy in two steps, where $\mathbf{u}_x, \mathbf{u}_y, \mathbf{u}_z$ are unit vectors along x-axis, y-axis, and z-axis. For line landmarks, we first apply an offset (x, y) to the original line $\mathbf{u}_z$ to obtain the translated line $\mathbf{l}(x, y)$. Then, we rotate the nearest point (x, y) using one 2 DoF rotation matrix $\mathbf{R}(\alpha, \beta)$ to obtain the final line $\mathbf{l}(\alpha, \beta, x, y)$. The 2 DoF rotation matrix $\mathbf{R}(\alpha, \beta)$ is defined as follows:

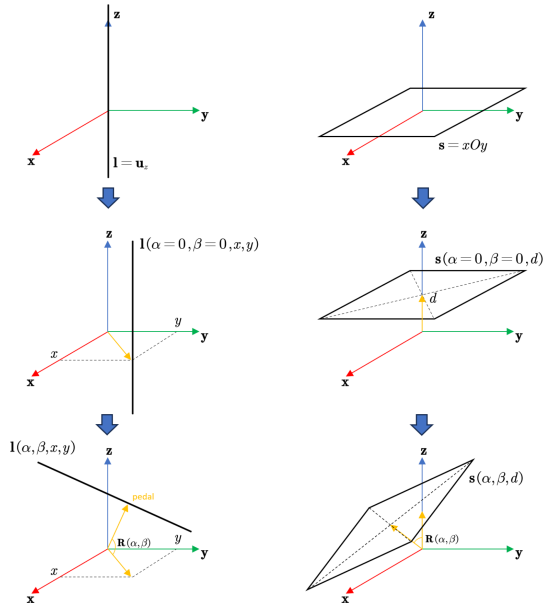


Fig. 3. Formulation of line and plane landmarks. On the left side, the line $\mathbf{l}(x, y)$ can be obtained through a translation (x, y), followed by the derivation of $\mathbf{l}(\mathbf{R}(\alpha, \beta), x, y)$ using a 2-DoF rotation $\mathbf{R}(\alpha, \beta)$. Similarly, on the right side, the plane $\mathbf{s}(d)$ is formulated by applying a translation d, and subsequently transformed into $\mathbf{s}(\mathbf{R}(\alpha, \beta), d)$ via rotation. A corresponding transformation always exists for any line or plane to formulate landmarks.

$$\mathbf{R}(\alpha, \beta) = \begin{bmatrix} \cos\beta & 0 & -\sin\beta \\ \sin\alpha\sin\beta & \cos\alpha & \sin\alpha\cos\beta \\ \cos\alpha\sin\beta & -\sin\alpha & \cos\alpha\cos\beta \end{bmatrix}. \quad (3)$$

For plane landmarks, the original plane xOy is translated along the z axis and rotated using a 2 DoF rotation matrix, resulting in the final plane $\mathbf{s}(\alpha, \beta, d)$.

The minimum parameterization of lines and planes necessitates residuals related to the pose and landmark. Though point-to-line and point-to-plane residuals are available in existing studies, they can not be applied directly on the parameterized forms of this study. Hence, it is essential to establish the mapping relations between our designed representations and the conventional point-normal form. The mappings are stated as follows:

$$\begin{bmatrix} \mathbf{n}_\mathcal{L} \\ \mathbf{c}_\mathcal{L} \end{bmatrix} = \begin{bmatrix} \mathbf{R}(\alpha, \beta)\mathbf{u}_z \\ \mathbf{R}(\alpha, \beta)(\mathbf{u}_x x + \mathbf{u}_y y) \end{bmatrix} \quad (4)$$

$$\begin{bmatrix} \mathbf{n}_\mathcal{S} \\ d_\mathcal{S} \end{bmatrix} = \begin{bmatrix} \mathbf{R}(\alpha, \beta)\mathbf{u}_z \\ d \end{bmatrix}. \quad (5)$$

Detailed residual formulations, utilizing the proposed line and plane representations, are presented in Appendix . The optimization process (map refinement) is introduced in Section IV-C and further illustrated in Appendix .

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

B. Global Map Merge

1) Map Merge Strategy: To this end, local maps are parameterized with lines and planes from multisession data. We refer to these local maps as *submaps* $\{\mathcal{M}_{s_i}\}$ in the remainder of this article. These submaps are located in different places and under different coordinate systems. Consequently, the global map merging process entails aligning the submaps to a common coordinate system, merging and eliminating redundant landmarks, and generating a globally consistent map that will serve as the base map for subsequent merges. We denote the merged global map as the base map $\mathcal{M}_b$. At the beginning of the merging process, we can select any submap to serve as the initial base map.

Typical global map merge methods first transform full laser scans into handcrafted or learning-based global descriptors for place retrieval. Then, point cloud registration is indispensable for relative transformation estimation, as introduced in Section II. Existing approaches often require extensive raw information for place recognition, such as the original point cloud or embedding feature maps, leading to significant data demands for learning and transmission. Additionally, these methods mostly focus on scan-to-scan place retrieval rather than utilizing multiframe information to encode a place. Single scan-based methods are more sensitive to displacement and lack robustness [20].

In this study, we propose registering submaps directly for global map merging *without* place recognition. Specifically, we adopt our previous work, G3Reg [78], for fast and robust registration. G3Reg utilizes graph theory, specifically the maximum clique (MC) algorithm, to prune correspondence outliers, with the landmark correspondences acting as graph nodes. However, each submap contains numerous landmarks (as detailed in Section IV-A), and solving the MC problem for such a large graph is computationally infeasible, resulting in an extremely long processing time.

To address this issue, we design a two-step method to reduce the problem size. First, we cluster the plane landmarks that lie on the same infinite plane into a single plane, particularly for those on large buildings and road surfaces. The line landmarks are maintained without clustering. Second, we partition the base map into blocks $\{\mathcal{G}b_p\}$ and the submap into blocks $\{\mathcal{G}s_q\}$ along the robot trajectory, instead of directly matching the submap to the base map. Each block comprises a host keyframe and a set of landmarks around it. These two steps make the map structure more compact for finding the maximum clique by reducing the number of nodes in a block, thereby leading to more efficient block registration.

2) Pairwise Block Global Registration: Estimating correct correspondences is crucial for global block registration. G3Reg proposes Gaussian ellipsoid models on landmarks, which consist of a centroid and its uncertainty represented by a pseudocovariance matrix, to construct translation and rotation invariant measurements (TRIMs). These TRIMs are then used to build a compatibility graph for correspondence outlier pruning.

In this study, the basic representations are parameterized infinite lines and planes without centroid modeling in Euclidean space. Such parameterization cannot support the construction of TRIMs used in the original G3Reg [78] for outlier pruning. To address this issue, we introduce the Grassmannian metric [79] to compute TRIMs and use the pairwise compatibility test to determine whether two correspondences are compatible. Specifically, lines and planes can be formulated as k-dimensional subspaces, where lines belong to 1-D subspaces and planes belong to 2-D subspaces. Their Grassmannian coordinates Y are written as follows:

$$Y = \begin{bmatrix} A & \frac{b}{\sqrt{\|b\|^2+1}} \\ 0 & \frac{1}{\sqrt{\|b\|^2+1}} \end{bmatrix} \in \mathbb{R}^{(n+1)\times(k+1)} \tag{6}$$

where A and b are the orthonormal basis and orthonormal displacement of the line nodes and plane nodes; n represents the dimension of Euclidean space ($n = 3$). Grassmannian metric $\mathrm{d}_{\text{Graff}}(Y_1, Y_2)$ can be defined to compute the distance of two subspaces

$$\begin{aligned} b_{02} &= b_2 - b_1 \\ Y_1' &= \begin{bmatrix} A_1 & 0 \\ 0 & 1 \end{bmatrix} \\ Y_2' &= \begin{bmatrix} A_2 & b_{02}/\sqrt{\|b_{02}\|^2+1} \\ 0 & 1/\sqrt{\|b_{02}\|^2+1} \end{bmatrix} \\ \{\sigma_i\} &= \mathrm{SVD}((Y_1')^T Y_2') \\ \mathrm{d}_{\text{Graff}}(Y_1, Y_2) &= \sum_i \arccos^2(\sigma_i). \end{aligned} \tag{7}$$

We denote all line and plane correspondences as $\mathcal{C}_{\mathcal{L}} = \{(k, l)\}$ and $\mathcal{C}_{\mathcal{S}} = \{(k, l)\}$, where k and l are the indices of landmarks in blocks $\mathcal{G}_{b_p}$ and $\mathcal{G}_{s_q}$ (blocks belong to the base map and the submap, respectively). The pairwise compatibility test for any two correspondences is formulated as follows:

$$\left| \mathrm{d}_{\text{Graff}}\left(Y_{k_i}, Y_{k_j}\right) - \mathrm{d}_{\text{Graff}}\left(Y_{l_i}, Y_{l_j}\right) \right| < \delta \tag{8}$$

where (k_i, l_i) and $(k_j, l_j) \in \mathcal{C}_{\mathcal{L}} \cup \mathcal{C}_{\mathcal{S}}$. Since the Grassmannian metric is $\mathrm{SE}(3)$-transformation invariant, we define a truncation threshold δ to determine whether both correspondences pass the compatibility test. If passed, an edge is connected between these correspondences in the compatibility graph. The maximum clique is then found using a fast exact parallel finder algorithm [80], where the nodes in this clique represent the putative line and plane correspondence inlier sets $\mathcal{C}_{\mathcal{L}}^*$ and $\mathcal{C}_{\mathcal{S}}^*$. Fig. 4 presents two cases of block registration for map merge in this study.

With the pruned correspondence results, we can establish a hybrid registration to estimate the relative poses between graphs (blocks), which is formulated as follows:

$$\begin{aligned} \min_{\mathbf{T}_{s_i}^{b_i}} & \sum_{(k,l)\in\mathcal{C}_{\mathcal{L}}^*} \rho\left(\| \left(\mathbf{I} - \mathbf{n}_k \mathbf{n}_k^T\right) \left(\mathbf{T}_{s_i}^{b_i} \mathbf{c}_l - \mathbf{c}_k\right) \|_{\mathbf{\Lambda}_{\mathcal{L}}}^2\right) \\ & + \sum_{(k,l)\in\mathcal{C}_{\mathcal{S}}^*} \rho\left(\|\mathbf{n}_k^T \left(\mathbf{T}_{s_i}^{b_i} \mathbf{c}_l - \mathbf{c}_k\right) \|_{\mathbf{\Lambda}_{\mathcal{S}}}^2\right) \end{aligned} \tag{9}$$

where $\mathbf{c} = b$, $\mathbf{n} = A$ for lines and $\mathbf{n} \in \mathrm{span}(A)^{\perp}$ for planes. The term $\mathbf{T}_{s_i}^{b_i}$ is the relative transformation between block $\mathcal{G}_{b_i}$ and

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

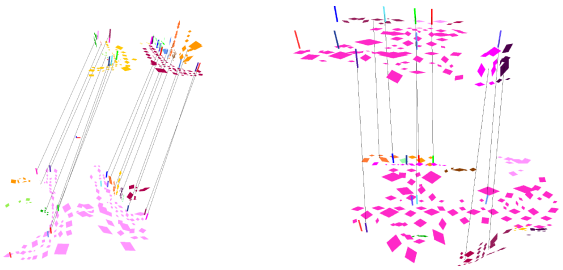


Fig. 4. Two registration cases at different places, using the proposed block registration method. We reduce the number of plane landmarks by clustering, thus speeding up the registration process for map merge. The lines between the clustered landmarks represent the data associations (correspondences) between the block in different coordinates.

block $\mathcal{G}_{s_i}$; $\rho(\cdot)$ denotes a robust kernel function, which further reduces the influence of potential correspondence outliers resulting from an inappropriate selection of the threshold δ, as discussed in [78]. At the start of iterations, the initial value of $\mathbf{T}_{s_i}^{b_i}$ is set to the identity transformation.

Furthermore, an optimization-based refinement is performed based on the k-nearest neighbors (kNN) search of landmarks to refine the relative pose estimation. Note that the landmark for refinement is the vectorized ones in Section IV-A2. The refinement is formulated as follows:

$$\min_{\mathbf{T}_{s_i}^{b_i}} \sum_{(k,l)\in\mathcal{N}_\mathcal{L}} \rho\left(\| \left(\mathbf{I}-\mathbf{n}_k\mathbf{n}_k^T\right)\left(\mathbf{T}_{s_i}^{b_i}\mathbf{p}_l-\mathbf{p}_k\right)\|_{\mathbf{\Lambda}_\mathcal{L}}^2\right) + \sum_{(k,l)\in\mathcal{N}_\mathcal{S}} \rho\left(\|\mathbf{n}_k^T\left(\mathbf{T}_{s_i}^{b_i}\mathbf{p}_l-\mathbf{p}_k\right)\|_{\mathbf{\Lambda}_\mathcal{S}}^2\right) \tag{10}$$

where (k,l) is the nearest-neighbor landmark pair in different blocks with respect to the initial pose estimated from block registration in (9). The terms $\mathbf{n}$ and $\mathbf{p}$ are with landmarks as demonstrated in Section IV-A2, not the clustered landmarks for block registration. Using original landmarks could improve the accuracy of registration compared to block registration. To enhance the robustness, the optimization problem is solved iteratively with updated landmark matching, which is the pipeline in the classical iterative closest point (ICP) [81]. Finally, we will obtain a set of relative poses $\mathcal{L}_\mathcal{O}^* := \{\mathbf{T}_{s_i}^{b_i}\}$ across sessions to serve as a candidate set for downstream modules.

3) Loop Outlier Rejection: Once all blocks have been successfully registered, we can align the submaps to the global coordinate system and proceed with the subsequent map refinement steps. However, false positive loop closures still remain due to several inevitable factors, such as inaccurate landmark parameterization and outliers in correspondences. We utilize a classical approach, PCM [32] to identify false loop candidates. Specifically, A function $C(\mathbf{T}_{s_k}^{b_k}, \mathbf{T}_{s_l}^{b_l})$ is designed to measure the consistency of block registration, based on the relative pose estimation in Section IV-B2, formulated as follows:

$$\begin{aligned}\delta\mathbf{T} &= (\mathbf{T}_{s_k}^{b_k})^{-1}\cdot\hat{\mathbf{T}}_{s_k}^{s_l}\cdot\mathbf{T}_{s_l}^{b_l}\cdot\hat{\mathbf{T}}_{b_l}^{b_k}\\ C(\mathbf{T}_{s_k}^{b_k}, \mathbf{T}_{s_l}^{b_l}) &= \left[\|\mathrm{Log}(\mathbf{R}(\delta\mathbf{T}))\|_2, \|\mathbf{p}(\delta\mathbf{T})\|_2\right]^T\end{aligned} \tag{11}$$

where $\hat{\mathbf{T}}_{s_k}^{s_l}$ and $\hat{\mathbf{T}}_{b_l}^{b_k}$ are from odometry trajectories in different sessions. If $C(\mathbf{T}_{s_k}^{b_k}, \mathbf{T}_{s_l}^{b_l})$ are small enough, the pair of relative pose $\mathbf{T}_{s_k}^{b_k}$ and $\mathbf{T}_{s_l}^{b_l}$ are considered to be consistent. Similarly, the problem of solving the internal-consistent set of relative poses is also a maximum clique problem [80]. The pruned set of relative poses (transformations) is denoted as $\mathcal{L}_\mathcal{O} = \{\mathbf{T}_{s_i}^{b_i}\}$, and will be used for subsequent PGO.

C. Map Refinement

1) Pose Graph Optimization: For each relative pose in $\mathcal{L}_\mathcal{O}$, we can construct a loop residual as

$$\mathbf{r}_{\mathcal{L}_\mathcal{O}}\left(\mathbf{T}_{b_i}^w, \mathbf{T}_{s_i}^w, \hat{\mathbf{T}}_{s_i}^{b_i}\right) = \begin{bmatrix}(\mathbf{R}_{b_i}^w)^T\left(\mathbf{p}_{s_i}^w-\mathbf{p}_{b_i}^w\right)-\hat{\mathbf{p}}_{s_i}^{b_i}\\ (\hat{\mathbf{R}}_{s_i}^{b_i})^T(\mathbf{R}_{b_i}^w)^T\mathbf{R}_{s_i}^w\end{bmatrix} \tag{12}$$

where w represents the world coordinate. In addition to the relative poses between blocks, the LiDAR odometry could also provide rigid transformations between consecutive keyframe poses, as illustrated in Section IV-A1. The odometry-based residual function is defined as

$$\mathbf{r}_\mathcal{O}\left(\mathbf{T}_{b_k}^w, \mathbf{T}_{b_l}^w, \hat{\mathbf{T}}_{b_l}^{b_k}\right) = \begin{bmatrix}(\mathbf{R}_{b_k}^w)^T\left(\mathbf{p}_{b_l}^w-\mathbf{p}_{b_k}^w\right)-\hat{\mathbf{p}}_{b_l}^{b_k}\\ (\hat{\mathbf{R}}_{b_l}^{b_k})^T(\mathbf{R}_{b_k}^w)^T\mathbf{R}_{b_l}^w\end{bmatrix} \tag{13}$$

where b_k and b_l are two adjacent keyframes. Within these residuals, the full PGO problem is formulated as

$$\begin{aligned}\min_{\mathbf{T}_b^w,\mathbf{T}_s^w} &\sum_{(b_k,b_l)\in\mathcal{O}_b}\rho\left(\|\mathbf{r}_\mathcal{O}\left(\mathbf{T}_{b_k}^w, \mathbf{T}_{b_l}^w, \hat{\mathbf{T}}_{b_l}^{b_k}\right)\|_{\mathbf{\Lambda}_\mathcal{O}}^2\right)\\ &+\sum_{(s_k,s_l)\in\mathcal{O}_s}\rho\left(\|\mathbf{r}_\mathcal{O}\left(\mathbf{T}_{s_k}^w, \mathbf{T}_{s_l}^w, \hat{\mathbf{T}}_{s_l}^{s_k}\right)\|_{\mathbf{\Lambda}_\mathcal{O}}^2\right)\\ &+\sum_{(b_i,s_i)\in\mathcal{L}_\mathcal{O}}\rho\left(\|\mathbf{r}_{\mathcal{L}_\mathcal{O}}\left(\mathbf{T}_{b_i}^w, \mathbf{T}_{s_i}^w, \hat{\mathbf{T}}_{s_i}^{b_i}\right)\|_{\mathbf{\Lambda}_{\mathcal{L}_\mathcal{O}}}^2\right)\end{aligned} \tag{14}$$

where $\mathcal{O}_b$ and $\mathcal{O}_s$ are the set of all odometry residuals of the base map and submap. $\mathbf{\Lambda}_\mathcal{O}$ and $\mathbf{\Lambda}_{\mathcal{L}_\mathcal{O}}$ are tunable parameters to describe the uncertainties. Empirically, $\mathbf{\Lambda}_\mathcal{O}$ is determined by the drift rate of applied odometry; $\mathbf{\Lambda}_{\mathcal{L}_\mathcal{O}}$ is insensitive to different datasets or scenarios, hence, a general threshold is acceptable. After solving the PGO, odometry drift will be preliminarily eliminated, and all keyframes of multisession data will be aligned more closely in the world coordinate. More specifically, We will merge the base map $\mathcal{M}_b$ and submap $\mathcal{M}_s$ at the data structure level, including the set of keyframes, the set of odometry residuals, landmarks, and all observations. The process of merging landmarks will be detailed as follows.

PGO improves mapping precision, i.e., the landmark states, by adjusting the poses. Following this, we perform landmark merging to reduce redundancy by merging landmarks that are close to each other. Specifically, we propose three criteria for merging plane landmarks: 1) the angle between the normal vectors of two planes is less than a threshold (set to 5° in the experiments); 2) the perpendicular distance of two planes is less than a threshold (set to 0.2 m in the experiments); 3) the two planes must have a large overlap, determined by whether the center of one landmark lies within the radius of another landmark. For line landmark merging, the thresholds for line merging are set as 5° for the angle between the directional vectors and 1.0 m for the perpendicular distance. The landmark merging

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

significantly reduces the number of landmarks and creates a more compact covisibility structure.

2) Bundle Adjustment: Drifted odometry could result in incorrect map vectorization, leading to inaccurate relative pose estimations between sessions that cannot be fully resolved by PGO. To address this, we propose a BA approach to improve map refinement by jointly adjusting landmarks and poses. Inspired by visual BA [82], we design the residual function between the observation in the keyframe and the corresponding landmark in the world. Geometrically, a line can be defined by two points. Thus, every line landmark has two point-to-infinite-line residuals

$$\mathbf{r}_{\mathcal{L}}\left(\mathbf{T}_f^w, \mathbf{l}_{\mathcal{L}}, \mathbf{f}_{\mathcal{L}}\right) = \begin{bmatrix} \mathbf{R}^T(\alpha,\beta)_{[2\times 3]}\left(\mathbf{I}-\mathbf{n}\mathbf{n}^T\right)\left(\mathbf{T}_f^w\hat{\mathbf{p}}_a - \mathbf{q}\right) \\ \mathbf{R}^T(\alpha,\beta)_{[2\times 3]}\left(\mathbf{I}-\mathbf{n}\mathbf{n}^T\right)\left(\mathbf{T}_f^w\hat{\mathbf{p}}_b - \mathbf{q}\right) \end{bmatrix} \in \mathbb{R}^4 \quad (15)$$

where $\mathbf{T}_f^w$ is the keyframe pose; $\mathbf{l}_{\mathcal{L}}$ is the line landmark and its point-direction form is $(\mathbf{n}, \mathbf{q})$; $(\cdot)_{[2\times 3]}$ represents the first two rows of the matrix because we use 2-D point-to-line distance other than general form such as $(\mathbf{I}-\mathbf{n}\mathbf{n}^T)(\mathbf{p}-\mathbf{q})$.

Similarly, a plane can be defined by three noncollinear points. The plane landmark and its observation have three point-to-infinite-plane residuals, formulated as follows:

$$\mathbf{r}_{\mathcal{S}}\left(\mathbf{T}_f^w, \mathbf{l}_{\mathcal{S}}, \mathbf{f}_{\mathcal{S}}\right) = \begin{bmatrix} \mathbf{n}^T\mathbf{T}_f^w\hat{\mathbf{p}}_a + d \\ \mathbf{n}^T\mathbf{T}_f^w\hat{\mathbf{p}}_b + d \\ \mathbf{n}^T\mathbf{T}_f^w\hat{\mathbf{p}}_c + d \end{bmatrix} \in \mathbb{R}^3 \quad (16)$$

where $\mathbf{l}_{\mathcal{S}}$ is the plane landmark and its distance-normal form is $(\mathbf{n}, d)$.

With (4), (5), (15), and (16), we can derive the Jacobian matrices with respect to line parameters $\langle \alpha, \beta, x, y\rangle$ and plane parameters $\langle \alpha, \beta, d\rangle$. Then the LiDAR BA can be formulated as follows:

$$\begin{aligned} \min_{\mathbf{T}_f^w, \mathbf{l}_{\mathcal{L}}, \mathbf{l}_{\mathcal{S}}} & \sum_{(f_k, f_l)\in\mathcal{O}} \rho\left(\|\mathbf{r}_{\mathcal{O}}\left(\mathbf{T}_{f_k}^w, \mathbf{T}_{f_l}^w, \hat{\mathbf{T}}_{f_l}^{f_k}\right)\|^2_{\mathbf{\Lambda}_{\mathcal{O}}}\right) \\ & + \sum_{(i,j)\in\mathcal{O}_{\mathcal{L}}} \rho\left(\|\mathbf{r}_{\mathcal{L}}\left(\mathbf{T}_{f_i}^w, \mathbf{l}_{\mathcal{L}_j}, \mathbf{f}_{\mathcal{L}}\right)\|^2_{\mathbf{\Lambda}_{\mathcal{L}_{i,j}}}\right) \\ & + \sum_{(i,j)\in\mathcal{O}_{\mathcal{S}}} \rho\left(\|\mathbf{r}_{\mathcal{S}}\left(\mathbf{T}_{f_i}^w, \mathbf{l}_{\mathcal{S}_j}, \mathbf{f}_{\mathcal{S}}\right)\|^2_{\mathbf{\Lambda}_{\mathcal{S}_{i,j}}}\right) \end{aligned} \quad (17)$$

where $\mathcal{O} = \mathcal{O}_b \cup \mathcal{O}_s$ represents the odometry residual set; $\mathcal{O}_{\mathcal{L}}$ and $\mathcal{O}_{\mathcal{S}}$ are point-to-line observation set and point-to-plane observation set, respectively. Please refer to the Appendix for the derivations of all Jacobians and more detailed illustrations. We also present a graphical illustration in Fig. 5 with factor graph representation. We will fix a certain frame to ensure the stability of the optimization. Considering the outliers in block registration caused by odometry drift, we do not integrate cross-session relative poses into the BA optimization.

The overall map refinement is performed in two steps: a pose-only approach (PGO) followed by a pose-landmark joint approach (BA), which offers two key advantages. First, the refinement process is conducted in a coarse-to-fine manner, which enhances the convergence of optimization. Performing BA directly without PGO may lead to convergence failure, as observed during tests using the SLIM system. Second, the optimized poses obtained from PGO serve as an initial guess for BA, thus improving the efficiency of the map refinement.

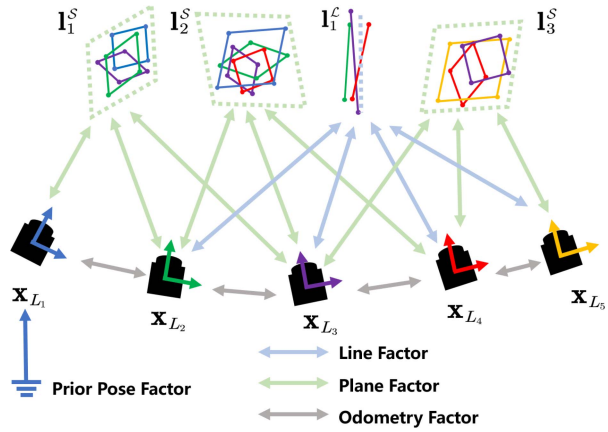


Fig. 5. Factor graph of the proposed LiDAR BA. Residuals are constructed from the odometric poses, point-to-line, and point-to-plane measurements. A prior pose factor is utilized to fix the global pose for optimization.

To this end, multisession maps are merged and refined sequentially, as detailed in Sections IV-B and IV-C. It is worth noting that these two modules utilize only parameterized lines and planes provided by the map vectorization in Section IV-A. Using such lightweight representations ensures memory efficiency while maintaining mapping accuracy. The experimental sections will validate the superiority of the proposed modules individually in terms of accuracy, efficiency, and generalizability across different datasets.

V. Map-Centric Marginalization

The dimensions of optimization will increase over time, which is a typical issue for long-term mapping. This growth leads to significant time costs for optimization. This section presents a map-centric marginalization that aims to maintain a factor graph with sparsified poses and unaltered landmarks as sessions increase. The proposed map-centric marginalization maintains global consistency while bounding the computational requirements for long-term mapping.

A. Motivation and Problem Formulation

In classical SLAM systems, *marginalization* is a crucial technique designed to prune the graph and preserve prior information without breaking the observability of the system [83]. The theoretical basis for this technique is the Schur complement [84]. However, the Schur complement generally introduces a "fill-in" of nonzero blocks in the Hessian matrix, significantly slowing down the nonlinear optimization. Nonlinear factor recovery [71], [72] approximates a new distribution of the original problem with fewer recovered factors. NFR is a classical method and plays a key role in the marginalization process of this study. The mathematical description of the NFR problem will be detailed as follows. Assuming that all the state variables, i.e., the robot poses and parameterized map landmarks, follow a Gaussian distribution: $p_o(\mathbf{s}) \sim \mathcal{N}(\mu_o, \mathbf{H}_o^{-1})$, where $\mathbf{X}$ encodes

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

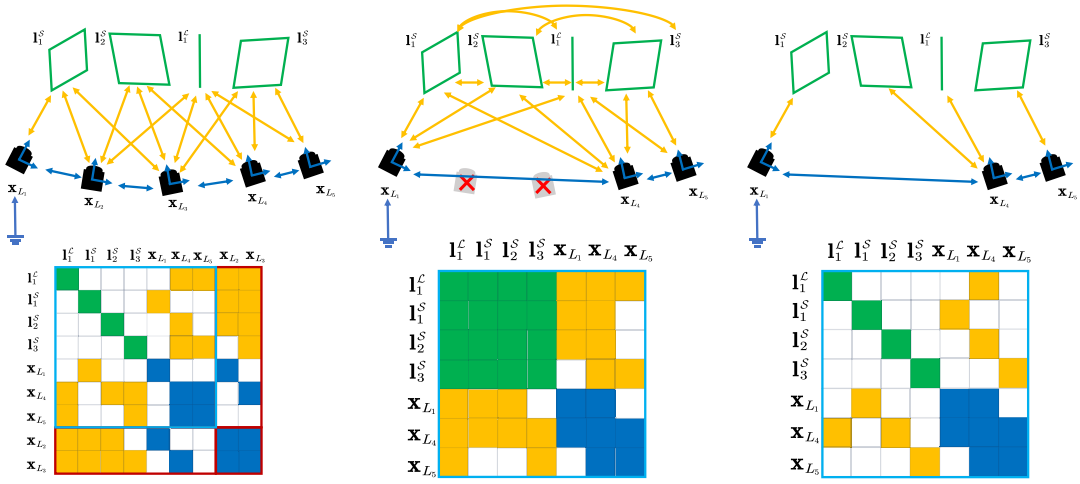


Fig. 6. Factor graph topology and its corresponding Hessian matrix. The left part shows the original factor graph and its Hessian matrix, where the green diagonal blocks represent the Hessian matrix for the landmarks. The light-blue box contains all the variables to be retained, while the red box contains the variables to be marginalized. The middle part shows the marginalized factor graph and the dense Hessian matrix, where the landmark part is no longer block-diagonal. Map-centric NFR aims to find the Hessian matrix on the right side. Our solution closely maintains the block diagonal structure; meanwhile approximates the probability distribution of the state variables represented by the intermediate problem, using only the sparse matrix block on the left side for calculations.

the state variables; μ_o and $\mathbf{H}_o$ are the current estimation and Hessian matrix (nearly equal to the inverse of covariance), respectively. The goal of NFR is to recover a new distribution $p_r(\mathbf{s}) \sim \mathcal{N}(\mu_r, \mathbf{H}_r^{-1})$ to approximate the original distribution $p_o(\mathbf{X})$ with a sparse topology. Essentially, this problem is a KLD minimization between the original distribution and the recovered distribution, described as follows:

$$D_{KL}\left(p_o\left(\mathbf{X}\right)||p_r\left(\mathbf{X}\right)\right) = \frac{1}{2}\left(\left\langle \mathbf{H}_r, \mathbf{H}_o^{-1}\right\rangle - \log\det\left(\mathbf{H}_r\right) + \|\mathbf{H}_r^{\frac{1}{2}}\left(\mu_r - \mu_o\right)\|^2 - d\right) \tag{18}$$

where d is the dimension of states.

Furthermore, we define the recovered residual as $\mathbf{r}_k(\mathbf{X}, \mathbf{z}_{r_k}) = (f(\mathbf{X}), \mathbf{z}_{r_k}, \mathbf{\Sigma}_{r_k}^{-1})$, where $\mathbf{z}_{r_k}$ is the kth *recovered observation*; $f(\mathbf{X})$ represents the residual function that encompasses $\mathbf{r}_\mathcal{O}$, $\mathbf{r}_\mathcal{L}$ and $\mathbf{r}_\mathcal{S}$; $\mathbf{\Sigma}_{r_k}^{-1}$ represents the kth *recovered information matrix*. Since modification of the current state variable estimation is irrational, the mean of the recovered distribution μ_r must equal to the mean of the original distribution μ_o. Accordingly, two conditions must be satisfied: $\mathbf{r}_k(\boldsymbol{\mu}_o, \mathbf{z}_{r_k}) = 0$ and $\boldsymbol{\mu}_o = \boldsymbol{\mu}_r$. Therefore, the original problem in (18) can be reformulated by excluding the third term and the constant fourth term

$$\begin{aligned} &\min_{\mathbf{\Sigma}_r} \frac{1}{2}\left(\left\langle \mathbf{J}_r^T\mathbf{\Sigma}_r^{-1}\mathbf{J}_r, \mathbf{H}_o^{-1}\right\rangle - \log\det\left(\mathbf{J}_r^T\mathbf{\Sigma}_r^{-1}\mathbf{J}_r\right)\right) \\ &\text{s.t.} \mathbf{\Sigma}_r^{-1} \succeq 0 \\ &\mathbf{J}_r = \begin{bmatrix} \mathbf{J}_{r_1} \\ \mathbf{J}_{r_2} \\ \vdots \\ \mathbf{J}_{r_K} \end{bmatrix} = \begin{bmatrix} \partial\mathbf{r}_1(\mathbf{X}, \mathbf{z}_{r_1})/\partial\mathbf{X} \\ \partial\mathbf{r}_2(\mathbf{X}, \mathbf{z}_{r_2})/\partial\mathbf{X} \\ \vdots \\ \partial\mathbf{r}_K(\mathbf{X}, \mathbf{z}_{r_K})/\partial\mathbf{X} \end{bmatrix} \\ &\mathbf{\Sigma}_r = \text{diag}(\mathbf{\Sigma}_{r_1}, \mathbf{\Sigma}_{r_2}, \ldots, \mathbf{\Sigma}_{r_K}) \end{aligned} \tag{19}$$

where $\mathbf{J}_r$ is all the concatenated jacobian blocks and $\mathbf{\Sigma}_r$ denotes the covariance matrix, which is the inverse of the information matrix to be estimated. It should be noted that if linearization point $\mathcal{X}$ and the observations $\mathbf{z}_{r_k}$ are known, $\mathbf{J}_r$ will be a constant matrix. Furthermore, if $\mathbf{J}_r$ is invertible, a closed-form solution exists [72], [85]

$$\mathbf{\Lambda}_{r_k} = \mathbf{\Sigma}_{r_k}^{-1} = \left\{\left(\mathbf{J}_r\mathbf{H}_o^{-1}\mathbf{J}_r^T\right)^{(k)}\right\}^{-1}. \tag{20}$$

Fig. 6 presents a graphical illustration of NFR and its corresponding graph topology. Specifically, our map-centric marginalization encompasses two primary steps, as detailed in the following section. First, a new topology is reconstructed from the original by keyframe removal; second, the information matrices of these residuals, which also serve as the solutions to the KLD problem, will be calculated as described in (20).

B. Two-Step Marginalization

1) Topology Reconstruction: First, we need to select keyframes for marginalization to obtain a new factor graph topology. A density-based downsampling method is applied to remove some keyframe poses in factor graphs. Specifically, a distance threshold is used to control the density of keyframe poses. To ensure the invertibility of the Jacobian matrix in (19), we propose the following criteria for connectivity specification.

1) Each landmark is connected only to the nearest keyframe pose.
2) All retained keyframes are connected by a minimum spanning tree, and $N-1$ keyframe-to-keyframe residuals are preserved (N is the number of retained keyframes).
3) Only one unique prior keyframe residual is needed.

The reconstructed Jacobian matrix is always square and invertible, meeting the requirements for (20). This is due to two reasons: the dimension of all keyframe-to-landmark residuals always equals the dimension of landmark states; the sum of

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

the dimensions of keyframe-to-keyframe residuals and the dimension of the prior keyframe pose residual exactly equals the dimension of retained keyframe states. It is worth noting that the topology reconstruction is performed after map refinement in Section IV-C. Therefore, even though the minimum spanning tree no longer includes any loop closures, the topology reconstruction does not affect the utilization of loop closures for map refinement.

Keyframe removal leads to information loss. Therefore, marginalization is needed to maintain the necessary information for mapping. The classical Schur-complement-based marginalization is formulated as follows. Assume that we have M line landmarks, N plane landmarks, P retained keyframes and Q marginalized keyframes. The dimensions are: retained state variables $r = M \times 4 + N \times 3 + P \times 6$; landmarks $l = M \times 4 + N \times 3$; retained frame $n = P \times 6$; marginalized state variables $m = Q \times 6$. Schur-complement-based marginalization is conducted when redundant keyframes are selected, providing the marginalized prior Hessian matrix. Given the Hessian matrix before marginalization $\mathbf{H}$, the covariance matrix $\mathbf{\Sigma}^m_{rr}$ after marginalization is derived as

$$\begin{aligned}\mathbf{H} &= \begin{bmatrix}\mathbf{H}_{rr} & \mathbf{H}_{rm}\\ \mathbf{H}^T_{rm} & \mathbf{H}_{mm}\end{bmatrix} \in \mathbb{R}^{(r+m)\times(r+m)}\\ \mathbf{\Sigma}^m_{rr} &= (\mathbf{H}^m_{rr})^{-1} = \left(\mathbf{H}_{rr} - \mathbf{H}_{rm}\mathbf{H}^{-1}_{mm}\mathbf{H}^T_{rm}\right)^{-1}\\ &= \begin{bmatrix}\mathbf{\Sigma}^m_{ll} & \mathbf{\Sigma}^m_{ln}\\ \mathbf{\Sigma}^{mT}_{ln} & \mathbf{\Sigma}^m_{nn}\end{bmatrix} \in \mathbb{R}^{r\times r}\end{aligned} \tag{21}$$

where $\mathbf{H}^m_{rr}$ is the Hessian matrix after the Schur complement, which refers to the $\mathbf{H}_o$ in (19). It is worth noting that $\mathbf{H}^m_{rr}$ no longer maintains its sparse property due to the Shur complement, and its inverse $\mathbf{\Sigma}^m_{rr}$ is completely dense. The marginalization aims to compute $\mathbf{\Sigma}^m_{rr}$, i.e., the matrix $\mathbf{H}^{-1}_o$, and determine all $\mathbf{\Lambda}_{r_i}$ in (20).

Marginalization with the Schur complement generally prunes states that include both landmarks and keyframes, as seen in previous visual mapping by Usenko et al. [76]. A relatively small dimension of state variables is retained in the marginalization. However, in this study, providing high-quality and complete landmarks is the goal for the SLIM system, and a map-centric marginalization approach is desired to maintain all map landmarks. Unlike previous works, our variable dimensions will range from hundreds of thousands to millions. The excessively large problem dimension leads to dual challenges of slow computation speed and insufficient memory when directly solving (21). This necessitates leveraging the inherent sparsity of the BA problem to achieve an efficient solution.

To address this challenge, we propose an *equivalent solution via sparse matrix operations*, which significantly improves the efficiency of marginalization and reduces memory consumption, making the SLIM system easily deployable for long-term operations. The relevant sparse matrix operations for efficient NFR will be detailed in the following section.

2) Efficient NFR: The purpose of efficient NFR is to avoid directly calculating $\mathbf{\Sigma}^m_{rr}$ and instead use the sparse matrix blocks of the Hessian matrix before marginalization $\mathbf{H}$ to accelerate the computation of (20). As previously mentioned, $\mathbf{\Sigma}^m_{rr}$ exhibits both high dimensionality and a dense structure. However, the sparsity inherent in the designed topology obviates the need to compute all blocks in $\mathbf{\Sigma}^m_{rr}$. For instance, the residual associated with a keyframe-to-landmark measurement with index k and the corresponding Jacobian matrix is

$$\mathbf{J}_{r_k} = \left[0, \ldots, \mathbf{J}^k_{\mathbf{l}_i}, 0, \ldots, 0, \mathbf{J}^k_{\mathbf{T}_j}, \ldots, 0\right]. \tag{22}$$

Given the covariance matrix $\mathbf{\Sigma}^m_{rr}$, the recovered information matrix is derived as

$$\begin{aligned}\mathbf{\Lambda}_{r_k} &= \mathbf{J}_{r_k}\mathbf{\Sigma}^m_{rr}\mathbf{J}^T_{r_k}\\ &= \mathbf{J}^k_{\mathbf{l}_i}\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{l}_i}\left(\mathbf{J}^k_{\mathbf{l}_i}\right)^T + \mathbf{J}^k_{\mathbf{l}_i}\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{T}_j}\left(\mathbf{J}^k_{\mathbf{T}_j}\right)^T +\\ &\quad \mathbf{J}^k_{\mathbf{T}_j}\left(\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{T}_j}\right)^T\left(\mathbf{J}^k_{\mathbf{l}_i}\right)^T + \mathbf{J}^k_{\mathbf{T}_j}\mathbf{\Sigma}^m_{\mathbf{T}_j\mathbf{T}_j}\left(\mathbf{J}^k_{\mathbf{T}_j}\right)^T.\end{aligned} \tag{23}$$

We can observe that the involved blocks are $\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{l}_i}$ in $\mathbf{\Sigma}^m_{ll}$, $\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{T}_j}$ in $\mathbf{\Sigma}^m_{ln}$ and $\mathbf{\Sigma}^m_{\mathbf{T}_j\mathbf{T}_j}$ in $\mathbf{\Sigma}^m_{nn}$. Notably, all the $\mathbf{\Sigma}^m_{\mathbf{l}_i\mathbf{l}_i}$ are diagonal blocks of $\mathbf{\Sigma}^m_{ll}$, so the off-diagonal blocks of $\mathbf{\Sigma}^m_{ll}$ do not need to be computed. Therefore, the essential blocks are the diagonal blocks of $\mathbf{\Sigma}^m_{ll}$, $\mathbf{\Sigma}^m_{ln}$ and $\mathbf{\Sigma}^m_{nn}$. Generally, we have $l \gg m > n$ considering the landmarks and keyframes in mapping. Therefore, the time and memory consumption of calculating the entire $\mathbf{\Sigma}^m_{ll}$ matrix is dominant, which is precisely what our sparse solution aims to reduce. We leverage two key identities derived from the Sherman-Morrison-Woodbury formula [86] to achieve both sparsity and computational efficiency. Specifically, for the inverse of matrix addition and partitioned matrix, we can obtain the following derivations:

$$(\mathbf{A} - \mathbf{U}\mathbf{C}\mathbf{V}^T)^{-1} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{U}(\mathbf{C}^{-1} - \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})^{-1}\mathbf{U}^T\mathbf{A}^{-1} \tag{24a}$$

$$\begin{bmatrix}\mathbf{A} & \mathbf{U}\\ \mathbf{V}^T & \mathbf{C}\end{bmatrix}^{-1} = \begin{bmatrix}\mathbf{X} & \mathbf{F}\\ \mathbf{G} & \mathbf{Y}\end{bmatrix} \tag{24b}$$

$$\mathbf{X} = (\mathbf{A} - \mathbf{U}\mathbf{C}^{-1}\mathbf{V}^T)^{-1} \tag{24c}$$

$$= \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{U}(\mathbf{C} - \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})^{-1}\mathbf{U}^T\mathbf{A}^{-1} \tag{24d}$$

$$\mathbf{Y} = (\mathbf{C} - \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})^{-1} \tag{24e}$$

$$\mathbf{F} = -\mathbf{A}^{-1}\mathbf{U}(\mathbf{C} - \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})^{-1} \tag{24f}$$

$$\mathbf{G} = -(\mathbf{C} - \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})^{-1}\mathbf{U}^T\mathbf{A}^{-1} \tag{24g}$$

then, combining with (21) and (24), we have

$$\mathbf{\Sigma}^m_{rr} = \mathbf{H}^{-1}_{rr} + \mathbf{H}^{-1}_{rr}\mathbf{H}_{rm}\left(\mathbf{H}_{mm} - \mathbf{H}^T_{rm}\mathbf{H}^{-1}_{rr}\mathbf{H}_{rm}\right)^{-1}\mathbf{H}^T_{rm}\mathbf{H}^{-1}_{rr}. \tag{25}$$

In our implementation, we strategically arrange the Hessian blocks, positioning all landmark-related blocks in the top left corner, marginalized blocks (frames) in the bottom right corner, and retained blocks in the center. This arrangement partitions the full Hessian matrix into nine distinct blocks, enabling us to reformulate the original problem presented in (21) as (26), (27), and (28). As depicted in Fig. 6, $\mathbf{H}_{ln}$, $\mathbf{H}_{nn}$, $\mathbf{V}_l$, $\mathbf{V}_n$ are sparse matrices and $\mathbf{H}_{ll}$ is a block-diagonal matrix. Fig. 7 presents a

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

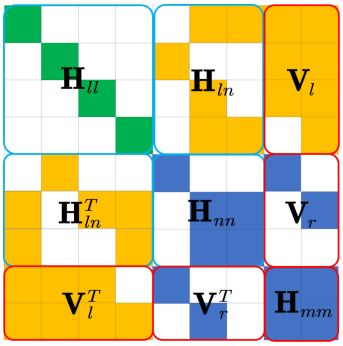


Fig. 7. Visualization for the Hessian matrix before marginalization. Noted that $\mathbf{H}_{ll}$ is block-diagonal and all of other blocks are sparse matrices in general. $\mathbf{H}_{ll}$ is much larger in dimensions than $\mathbf{H}_{nn}$ and $\mathbf{H}_{mm}$. Therefore, in the implementation, we only store its diagonal blocks, while all other matrices are saved using a sparse matrices.

graphical understanding of the nine blocks

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{ll} & \mathbf{H}_{ln} & \mathbf{V}_l \\ (\mathbf{H}_{ln})^T & \mathbf{H}_{nn} & \mathbf{V}_n \\ (\mathbf{V}_l)^T & (\mathbf{V}_n)^T & \mathbf{H}_{mm} \end{bmatrix} \in \mathbb{R}^{(r+m)\times(r+m)} \tag{26}$$

$$\mathbf{H}_{rr} = \begin{bmatrix} \mathbf{H}_{ll} & \mathbf{H}_{ln} \\ (\mathbf{H}_{ln})^T & \mathbf{H}_{nn} \end{bmatrix} \in \mathbb{R}^{r\times r} \tag{27}$$

$$\mathbf{H}_{rm} = \begin{bmatrix} \mathbf{V}_l \\ \mathbf{V}_n \end{bmatrix} \in \mathbb{R}^{r\times m}. \tag{28}$$

From (25), we identify two crucial matrices: $\mathbf{H}_{rr}^{-1}$ and $\mathbf{R} = (\mathbf{H}_{mm} - (\mathbf{H}_{rm})^T(\mathbf{H}_{rr})^{-1}\mathbf{H}_{rm})^{-1}$. The block-diagonal property of $\mathbf{H}_{ll}$ allows for the efficient parallel computation of its inverse, which is directly relevant to calculating $\mathbf{H}rr^{-1}$. Similarly, the computation of $\mathbf{R}$ can be streamlined by exploiting the $\mathbf{H}_{ll}$ structure. In the subsequent discussion, we will outline a procedure to obtain $\mathbf{R}$ using a series of sparse matrix operations, ensuring low memory consumption and high computational speed.

Let us define $\mathbf{D} = (\mathbf{H}_{ll})^{-1}$ (block-diagonal and positive definite), $\mathbf{P} = (\mathbf{H}_{ll})^{-1}\mathbf{H}_{ln}$ (sparse) and $\mathbf{Q} = (\mathbf{H}_{nn} - (\mathbf{H}_{ln})^T(\mathbf{H}_{ll})^{-1}\mathbf{H}_{ln})^{-1}$ (dense and positive definite). We can derive more compact forms to express $\mathbf{H}_{rr}^{-1}$ (block-diagonal and positive definite) and $\mathbf{\Sigma}_{rr}^m$ (dense and positive definite), as follows:

$$\mathbf{H}_{rr}^{-1} = \begin{bmatrix} \mathbf{D} + \mathbf{PQP}^T & -\mathbf{PQ} \\ -\mathbf{Q}^T\mathbf{P}^T & \mathbf{Q} \end{bmatrix} \tag{29}$$

$$\mathbf{\Sigma}_{rr}^m = \mathbf{H}_{rr}^{-1} + \mathbf{H}_{rr}^{-1}\begin{bmatrix} \mathbf{V}_l \\ \mathbf{V}_n \end{bmatrix}\mathbf{R}\begin{bmatrix} \mathbf{V}_l^T & \mathbf{V}_n^T \end{bmatrix}\mathbf{H}_{rr}^{-1}. \tag{30}$$

Given the sparsity of matrices $\mathbf{V}_l$ and $\mathbf{V}_n$, we can efficiently compute the following *intermediary matrices:* $\mathbf{PQ}$, $\mathbf{DV}_l$, $\mathbf{P}^T\mathbf{V}_l$, $\mathbf{PQV}_n$, and $\mathbf{QV}_n$. Therefore, computation of $\mathbf{R}$ will involve several sparse matrix operations, as follows:

$$\begin{aligned} \mathbf{R}^{-1} &= \mathbf{H}_{mm} - (\mathbf{H}_{rm})^T(\mathbf{H}_{rr})^{-1}\mathbf{H}_{rm} \in \mathbb{R}^{m\times m} \\ &= \mathbf{H}_{mm} - \begin{bmatrix} \mathbf{V}_l^T & \mathbf{V}_n^T \end{bmatrix}\begin{bmatrix} \mathbf{D} + \mathbf{PQP}^T & -\mathbf{PQ} \\ -\mathbf{Q}^T\mathbf{P}^T & \mathbf{Q} \end{bmatrix}\begin{bmatrix} \mathbf{V}_l \\ \mathbf{V}_n \end{bmatrix} \\ &= \mathbf{H}_{mm} - \mathbf{V}_l^T\mathbf{DV}_l - \mathbf{V}_l^T\mathbf{PQP}^T\mathbf{V}_l \\ &\quad + \left(\mathbf{V}_l^T\mathbf{PQV}_n\right)^T + \left(\mathbf{V}_l^T\mathbf{PQV}_n\right) - \mathbf{V}_n^T\mathbf{QV}_n \end{aligned} \tag{31}$$

which enables high-efficiency computing by leveraging advanced packages such as Eigen [87] and cuBLAS [88].

Based on the Schur complement, $\mathbf{R}$ represents the marginalized covariance matrix of the marginalized frames. The matrix $\mathbf{R}$ is inherently positive definite, ensuring its inverse exists. We leverage QR decomposition to derive $\mathbf{R}$ from its inverse, $\mathbf{R}^{-1}$. Furthermore, we exploit the inherent symmetry in the second term of $\mathbf{\Sigma}_{rr}^m$ in (25), which can reduce the computational burden by calculating only one part. Specifically, We apply a Cholesky decomposition on $\mathbf{R}$ to obtain $\mathbf{R} = \mathbf{R}_c(\mathbf{R}_c)^T$, and we have

$$\begin{aligned} \mathbf{W} &= \begin{bmatrix} \mathbf{D} + \mathbf{PQP}^T & -\mathbf{PQ} \\ -\mathbf{Q}^T\mathbf{P}^T & \mathbf{Q} \end{bmatrix}\begin{bmatrix} \mathbf{V}_l \\ \mathbf{V}_n \end{bmatrix}\mathbf{R}_c \\ &= \begin{bmatrix} \mathbf{D}\left(\mathbf{V}_l\mathbf{R}_c\right) + \mathbf{PQ}\left[\left(\mathbf{P}^T\mathbf{V}_l - \mathbf{V}_n\right)\mathbf{R}_c\right] \\ -\mathbf{Q}\left[\left(\mathbf{P}^T\mathbf{V}_l - \mathbf{V}_n\right)\mathbf{R}_c\right] \end{bmatrix} \\ \mathbf{W}_l &= \mathbf{DV}_l\mathbf{R}_c + \mathbf{PQ}\left(\mathbf{P}^T\mathbf{V}_l - \mathbf{V}_n\right)\mathbf{R}_c \\ \mathbf{W}_n &= -\mathbf{Q}\left(\mathbf{P}^T\mathbf{V}_l - \mathbf{V}_n\right)\mathbf{R}_c \\ \mathbf{\Sigma}_{rr}^m &= \begin{bmatrix} \mathbf{D} + \mathbf{PQP}^T & -\mathbf{PQ} \\ -\mathbf{Q}^T\mathbf{P}^T & \mathbf{Q} \end{bmatrix} + \begin{bmatrix} \mathbf{W}_l\mathbf{W}_l^T & \mathbf{W}_l\mathbf{W}_n^T \\ \mathbf{W}_n\mathbf{W}_l^T & \mathbf{W}_n\mathbf{W}_n^T \end{bmatrix} \end{aligned} \tag{32}$$

in which *intermediary matrices* in these equations can be reused directly as intermediate results.

As aforementioned, we only need to compute $\mathbf{\Sigma}_{ln}^m = -\mathbf{PQ} + \mathbf{W}_l\mathbf{W}_n^T$ and $\mathbf{\Sigma}_{nn}^m = \mathbf{Q} + \mathbf{W}_n\mathbf{W}_n^T$ and the diagonal blocks of $\mathbf{\Sigma}_{ll}^m = \mathbf{D} + \mathbf{PQP}^T + \mathbf{W}_l\mathbf{W}_l^T$. Notably, the computation of the $\mathbf{D} + \mathbf{PQP}^T + \mathbf{W}_l\mathbf{W}_l^T$ can be accelerated through parallel processing, further enhancing efficiency. The remaining two matrices, characterized by their lower dimensionality, can be computed directly. The situation we discussed earlier only considers the Jacobian matrix for keyframe-to-landmark residual. The Jacobian matrices for keyframe-to-keyframe and prior pose measurements depend solely on $\mathbf{Q} + \mathbf{W}_n\mathbf{W}_n^T$, ensuring the feasibility of our proposed sparse solution.

Finally, within the each single Jacobian block $\mathbf{J}_{r_k}$ and the essential blocks of $\mathbf{\Sigma}_{rr}^m$, i.e., $\mathbf{H}_o^{-1}$, all the recovered information matrices $\{\mathbf{\Lambda}_{r_k}\}$ could be calculated with (23) in parallel. So far, we have introduced the construction of all recovered observations after marginalization, resulting in a sparse factor graph that is very close to the dense factor graph. The experimental section will demonstrate the necessity of map-centric marginalization and the effectiveness of our proposed NFR.

VI. Experiments

We conduct real-world experiments to evaluate the performance of the proposed SLIM system. We first introduce the experimental setup in Section VI-A. The following tests are

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

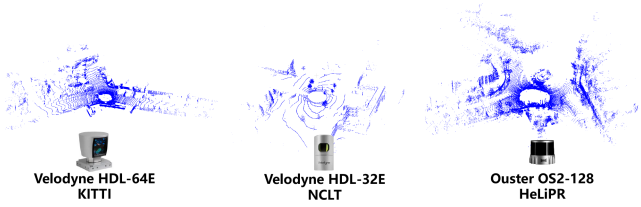


Fig. 8. LiDAR sensors and scans in KITTI, NCLT, and HeLiPR. The origin of each scan, corresponding to the sensor frame, is positioned at the center of the LiDAR point clouds. The properties of point clouds, such as density, vary with different sensor configurations. The front-end map vectorization module can still parametrize the point clouds to lines and planes, ensuring the functionality of the subsequent map refinement and maintenance modules.

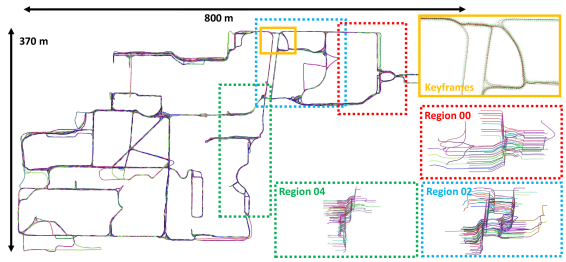
designed to validate the four capabilities of SLIM and to address the motivations outlined in Section I.

1) *Accuracy:* Accuracy is the main concern for robotic localization and mapping. The accuracy of SLIM is quantitatively evaluated in Section VI-B, with comparisons to other advanced methods.
2) *Lightweightness:* In Section VI-C, we compare the requirements for storage space (memory consumption) with other map representations.
3) *Scalability:* The designed marginalization is validated in Section VI-D individually with multisession maps.
4) *Localizability:* Section VI-E demonstrates that the maps from SLIM can be reused for online robot localization.

A. Experimental Set-Up

Three real-world datasets are employed for comprehensive validation and quantitative evaluation: KITTI [89], NCLT [90], and HeLiPR [9]. The KITTI dataset is collected by a vehicle equipped with a Velodyne HDL-64E. We use Sequence 00-10 for experimental validation, although some sequences are not entirely in urban environments. We employ KISS-ICP [14] to obtain LiDAR odometry as prior poses. Each sequence is partitioned into multiple sessions with overlapping regions before proceeding with our map merging pipeline. NCLT is also a well-known dataset suitable for the evaluation of multisession mapping [67]. Multiple sessions are collected from a mobile platform equipped with a Velodyne HDL-32E. Due to the rotational motion of the platform, we use FAST-LIO2 [4] for point cloud undistortion and LiDAR odometry generation. We selected six regions as our benchmark, with each region containing between 25 and 60 sessions, meaning the robot travels to the location more than 25 times.

Both KITTI and NCLT have nearly a decade of history. For the recent HeLiPR [9], similar to the NCLT dataset, we partition 32 overlapping regions as our benchmark, with each region having around ten sessions. A fast G-ICP [91] is employed as LiDAR odometry on the HeLiPR dataset. The reason for using various LiDAR odometry methods is that LiDAR sensing varies across the three datasets, as shown in Fig. 8. Thus, we select a suitable odometry configuration for each dataset. Overall, these datasets are collected in different countries worldwide and on different platforms, thus verifying the generalization ability of the proposed SLIM system.



(a) NCLT

Fig. 9. Multisession datasets in NCLT. We employ the regions, a type of large-scale submaps, to better manage urban mapping. The zoomed-in views in the red, green and blue boxes provide trajectories of multisession robot travels, visualized with varying heights and colors. The yellow boxes provide zoomed keyframe views of poses on the x–y plane. (a) NCLT.

As mentioned above, we partition the large-scale urban areas into regions for evaluation. Essentially, the *region* represents a type of submap for long-term management, which differs from the concept of *session*. More specifically, the former is in the spatial domain, while the latter is in the temporal domain. We present the visualization results of the multisession data and the regions in Fig. 9 for a better illustration. Fig. 10 also shows the mapping results in different regions. For visualization, a plane landmark is represented by a rhombus formed by selecting four points. These four control points are determined by computing the PCA of the weighted sum of the observed points within each landmark.

B. Mapping Accuracy

This section validates that SLIM can provide a globally consistent map from multisession data. We first compare SLIM with other advanced mapping systems on KITTI, using trajectory accuracy for quantitative evaluation. Then, we demonstrate long-term map merging and consistent mapping with incremental map sessions on the HeLiPR dataset.

1) Mapping Accuracy on KITTI Dataset: Although KITTI covers multiple scenarios beyond urban environments, such as rural and highway, the proposed SLIM system can still provide line and plane maps. Since KISS-ICP lacks loop closure capability, SLIM autonomously searches for overlapping regions of each submap, estimates relative poses, and then performs subsequent PGO and BA. In practical deployments, robots are frequently equipped with additional global localization functions, such as GPS, to facilitate map merging. The integration of such information provides strong global pose priors for the mapping system, effectively reducing the complexities of global map merging.

Regarding the evaluation metrics, we compute the absolute rotation and translation error between the keyframes in our global map and the corresponding frames in the ground truth trajectory. The benchmark results are summarized in Table I, with partial results from BALM2 [12]. The overall results indicate that the advanced PIN-SLAM [19] achieves the best performance, while BALM2 [12] and proposed SLIM achieve competitive performance on certain sequences. We observe that SLIM performs poorly on Sequences 01 and 09

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

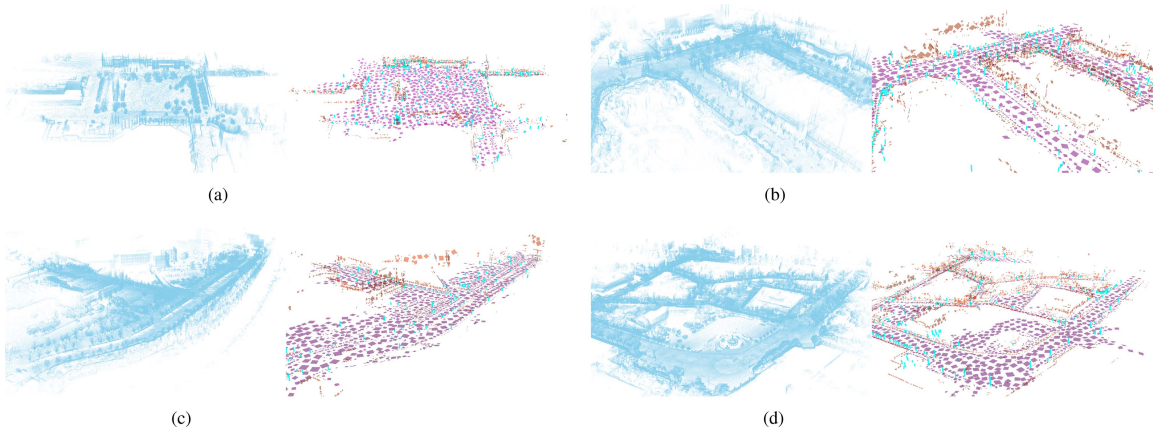


Fig. 10. Visualization of maps on NCLT and HeLiPR. We present the downsampled point cloud maps and the SLIM-generated maps from the same perspective view. The point cloud maps are built by accumulating LiDAR points on the robot poses estimated by SLIM. SLIM provides line and plane-based maps that are of high consistency (Section VI-B) and can be reused for robot localization (Section VI-E). More importantly, the map is much more lightweight (Section VI-C) and scalable for long-term use (Section VI-D). (a) NCLT Region 04. (b) HeLiPR DCC Region 05. (c) HeLiPR KAIST Region 00. (d) HeLiPR Town Region 05.

TABLE I
ABSOLUTE TRAJECTORY ERROR (RMSE, METERS) ON KITTI

Sequence	00	01	02	03	04	05	06	07	08	09	10	Mean
Eigen-Factor [17]	1.02	1.94	5.28	0.70	0.82	0.84	0.31	0.43	2.80	1.98	0.99	1.55
CT-ICP [92]	1.68	2.25	4.06	0.67	0.67	0.76	0.34	0.40	2.52	**0.91**	0.83	1.40
MULLS [13]	1.09	1.96	5.42	0.74	0.89	0.97	0.31	0.44	2.93	2.12	1.13	1.63
BALM [46]	0.96	1.90	5.21	0.68	0.75	0.72	0.28	0.40	2.72	1.75	0.92	1.48
Plane Adjustment [93]	0.86	1.84	5.08	**0.57**	0.64	0.66	0.23	0.31	2.63	1.50	0.80	1.37
BAREG [94]	0.89	1.88	5.12	0.65	0.70	0.69	0.24	0.35	2.68	1.59	0.88	1.42
BALM2 [12]	0.84	**1.83**	5.06	0.57	0.64	0.62	**0.21**	**0.30**	2.59	1.48	**0.78**	1.34
LONER[1] [54]	14.13	×	69.68	5.19	1.22	6.65	1.00	×	17.86	6.88	7.57	NA
PIN-LO [19]	5.6	4.3	9.3	0.7	**0.1**	1.7	0.5	0.5	3.0	1.8	0.8	2.6
PIN-SLAM [19]	**0.8**	4.3	2.1	0.7	**0.1**	**0.3**	0.4	**0.3**	**2.1**	1.2	0.8	**1.2**
SLIM (Ours)	0.95	3.72	**1.99**	0.79	0.28	0.61	0.24	0.33	2.31	3.12	1.07	1.40

The bold values are with the minimum errors.

due to insufficient roadside features, such as streetlights and trees.

Among these comparisons, only the proposed SLIM, BALM2 [12], and PLC-LiSLAM [47] achieve feature-level LiDAR BA. BALM2 primarily focuses on state estimation rather than long-term mapping, as it lacks an explicit representation of landmarks. Additionally, BALM2 uses dense points for data association and residual construction and relies on a second-order solver. In contrast, PLC-LiSLAM uses planes, lines, and cylinders as landmarks with explicit representation. However, its landmark representation is not minimal, which necessitates additional design efforts to ensure correct state updates and maintain the sparsity of the nonlinear optimization problem. Our proposed SLIM employs a minimal landmark representation, ensuring that the corresponding optimization naturally exhibits a sparse structure. Furthermore, we design and validate the map merge and marginalization capabilities within the SLIM system, enhancing its overall functionality.

2) Incremental Mapping on HeLiPR Dataset: Compared to the KITTI dataset, the recently published HeLiPR dataset is more suitable for validating long-term mapping. This is due to the inter-LiDAR sequences in HeLiPR, which naturally verify map merging and refinement as sessions increase at the same location. Specifically, the HeLiPR dataset encompasses sequences collected in four different scenarios: DCC, KAIST, Roundabout, and Town, with three sequences in each scenario. To obtain multiple submaps with overlapping regions, we selected 32 regions as our benchmark, with each region containing around ten sessions. The SLIM system sequentially merges multiple submaps onto the base map, achieving incremental mapping from multisession data. As depicted in Algorithm 1, SLIM conducts PGO and BA when a new session is merged, allowing for evaluation of the merged trajectory accuracy. We also compute the trajectory accuracy of the current submap to be merged. The heights of ground truth trajectories are not stable in some sequences of HeLiPR, so we evaluate trajectory accuracy along the x and y only.

We present the experimental results of 20 sequences quantitatively in Fig. 11. The SLIM system can maintain the consistency of the global map as new maps are added, even if the new maps are aligned with poor prior poses. Notably, the error of BA is generally less than PGO and the original LiDAR odometry, demonstrating the effectiveness of the proposed map refinement in Section IV-C. We also present qualitative results by

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

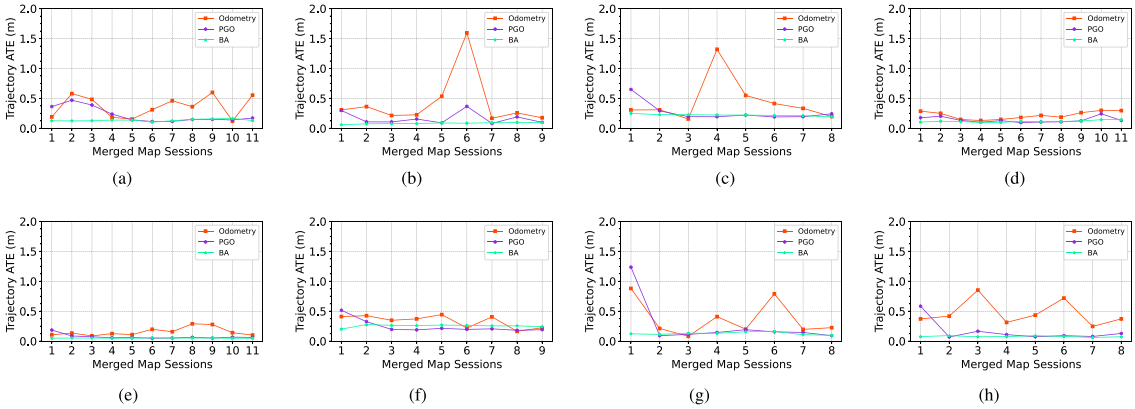


Fig. 11. Quantitative results of mapping accuracy on HeLiPR. The evaluation metric is the RMSE of absolute trajectory error. The overall results indicate that: 1) PGO (purple) and BA (cyan) effectively reduce errors and improve the consistency of the global map; 2) in certain cases, inaccurate submaps degrade the refinement results of PGO, while BA can almost entirely mitigate such impacts, demonstrating the necessity and effectiveness of our proposed BA. (a) Roundabout Region 00. (b) Roundabout Region 01. (c) Town Region 00. (d) Town Region 01. (e) DCC Region 00. (f) DCC Region 01. (g) KAIST Region 00. (h) KAIST Region 01.

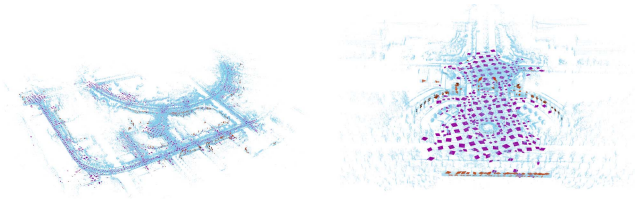


Fig. 12. Demonstration on a university campus on M2DGR. For comparison, the point cloud maps are built by accumulating LiDAR points based on the robot poses estimated by SLIM. (a) Multisession map on M2DGR. (b) Map view at the campus gate.

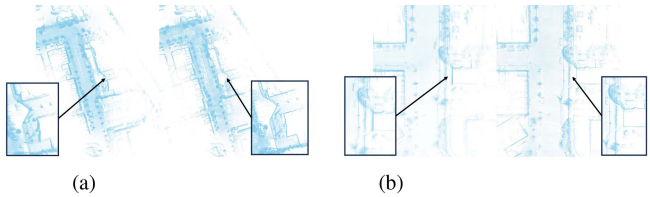


Fig. 13. Visualizing the BA effectiveness in HeLiPR. The point cloud maps are built by accumulating LiDAR points on robot poses provided by SLIM. The poses on the left side are obtained before the LiDAR BA, i.e., only using the PGO. After applying the BA, the poses on the right have fewer drifts, resulting in high-precision maps in zoomed-in views. This figure demonstrates the effectiveness of the proposed LiDAR BA on parameterized representations. (a) DCC Region 06. (b) Roundabout Region 00.

applying our proposed BA in Fig. 13, verifying the necessity of using BA for high-consistency mapping. We observe unstable results after applying PGO due to low-quality relative poses from block registration when odometry poses are not accurate enough. In summary, given multisession maps, the map refinement modules can effectively reduce pose errors, i.e., improve mapping accuracy, by utilizing our proposed map merging and refinement.

One key capability of our framework is map merging, as described in Section IV-B, which is harder for quantitative evaluation compared to mapping accuracy. All the input submaps in this section are within their own coordinates when generated. The map merge module can align them into one global frame, resulting in the error reduction shown in Fig. 11. The video in the supplementary materials also presents the map merge process to facilitate understanding.

In addition to the KITTI, NCLT, and HeLiPR, we also test the SLIM system qualitatively on the M2DGR dataset [95].We present the qualitative mapping result in Fig. 12. The SLIM system achieves map merging and refinement on the multisession data provided by M2DGR, demonstrating its effectiveness in structural environments.

C. Memory Efficiency

Urban environments cover large spaces and various scenarios, so memory efficiency is a major concern for mobile robots. In this section, we evaluate the memory efficiency of SLIM compared to several conventional mapping systems in terms of storage size (memory consumption). The SLIM system utilizes lines and planes as basic front-end representations. The stored map can also contain graph structures that encode poses and covisible information. If the map is solely designed for localization without such structures, we denote it with an (L) label for clarity. The first seven methods (from Eigen-Factor [17] to BALM2 [12]) in Table I utilize point clouds as explicit representation, resulting in dense point clouds as generated output maps.

The memory consumption of different maps is first evaluated on the KITTI dataset. The comparisons include both explicit representations, such as sampled point clouds (with r as the sampling radius), and recent implicit neural representations [19], [53], [54], [55]. Partial results are from previous studies [8], [19], and all are summarized in Table II. Our lightweight map requires significantly less storage than other mapping systems, demonstrating its superiority in memory efficiency for urban environments. Specifically, SLIM (L) only needs less than 0.5 Megabyte (MB) to cover a travel length of over 3 kilometers (km), with a density of approximately 130 KB/km. If the full SLIM map is needed for updates, the system uses less than 15 MB per sequence to store on disks.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

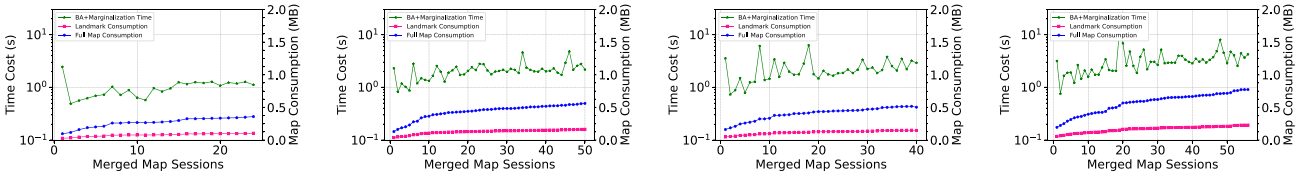


Fig. 14. Quantitative results of map consumption of SLIM and the time cost for map refinement. The green line represents "BA+Marginalization Time," the pink line is "Landmark Consumption," and the blue line is "Full Map Consumption." The results on NCLT indicate that 1) the provided map is lightweight and only uses low storage consumption; 2) the consumption and time cost remain stable as the number of sessions increases. (a) NCLT Region 00. (b) NCLT Region 01. (c) NCLT Region 02. (d) NCLT Region 03.

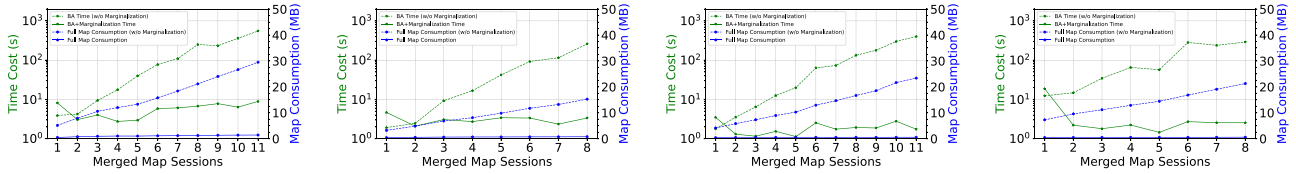


Fig. 15. Quantitative results of using and without using marginalization on HeLiPR. The blue solid line represents "Full Map Consumption," the blue dashed line is "Full Map Consumption (w/o Marginalization)," the green solid line represents "BA+Marginalization Time" and the green dashed line is "BA Time (w/o Marginalization)". If the marginalization is enabled, the time cost and map consumption are bounded. More specifically, the BA and marginalization cost less than 10 seconds. The map consumes less than 1 MB for one region. (a) Roundabout Region 00. (b) Town Region 00. (c) DCC Region 00. (d) KAIST Region 00.

TABLE II
MEMORY CONSUMPTION (MEGA BYTE, MB) ON KITTI

Sequence (Length)	00 (3.7 Km)	05 (2.2 Km)	08 (3.2 Km)
Raw point cloud	13624.2	8284.7	12214.1
Point cloud (r=0.1m)	2831.46	1852.96	2769.22
Point cloud (r=0.3m)	52.03	32.58	70.41
Surfel map [96]	887.7	512.6	835.7
Mesh map [97]	2032.9	1317.4	1894.1
VDB TSDF map [98]	748.1	434.6	958.6
SHINE map [53]	160.6	114.2	189.9
LONER^2 [54]	150.1	150.1	150.1
NeRF-LOAM^3 [55]	953.61	800.69	1424.9
PIN-SLAM [19]	102.1	66.3	138.8
SLIM	**12.9**	**7.2**	**9.2**
SLIM (L)	**0.5**	**0.3**	**0.4**

The bold values are with the minimum errors.

The memory consumption on NCLT and HeLiPR is presented in Figs. 14 and 15, respectively. In Fig. 14, each region requires less than 1 MB to store a full map structure. Similar results are found in Fig. 15. In summary, the built maps require low storage consumption on these two datasets, consistent with the maps generated from KITTI.

It is worth noting that the "efficiency" and "lightweight" in this study differ from those in recent LiDAR odometry [4], [99]. The two odometry systems focus on short-term and computationally efficient LiDAR mapping, while the SLIM is designed for long-term and memory-efficient mapping. High memory efficiency could reduce data transmission load in multirobot systems for distributed operations, such as crowd-sourced urban mapping.

D. Scalability for Long-Term Mapping

Sections VI-B and VI-C demonstrate that the SLIM system provides not only accurate but also memory-efficient maps in urban environments. The map merge capability is also verified in Section VI-B. As discussed in Section I, one motivation for SLIM is scalability, which is crucial for long-term mapping to maintain global consistency and bound map sizes as map sessions increase. Scalability is particularly important to ensure that map refinement, especially for high-dimensional LiDAR BA, remains controllable.

We verify the scalability of SLIM on NCLT and HeLiPR. For NCLT, Fig. 14 shows that both map consumption and time cost remain within certain limits even with 50 sessions in the same region. This satisfying performance is primarily attributed to the marginalization module. To verify this, we disable the marginalization and conduct tests on HeLiPR. In Fig. 15, if marginalization is disabled, we observe that map optimization time exhibits near-exponential growth, while map storage consumption increases linearly with the number of sessions. Overall, the full system maintains controllable costs on both NCLT and HeLiPR, verifying the scalability of SLIM. We do not provide experimental results on NCLT, as disabling marginalization will extend submap merging time to tens of minutes or even hours.

The proposed marginalization consists of two steps: topological reconstruction (Section V-B1) followed by map-centric NFR (Section V-B2), and the latter is one of the key contributions of this study. One might argue that only topological reconstruction or other advanced keyframe sampling techniques could maintain a sparse map structure for long-term mapping, making the

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

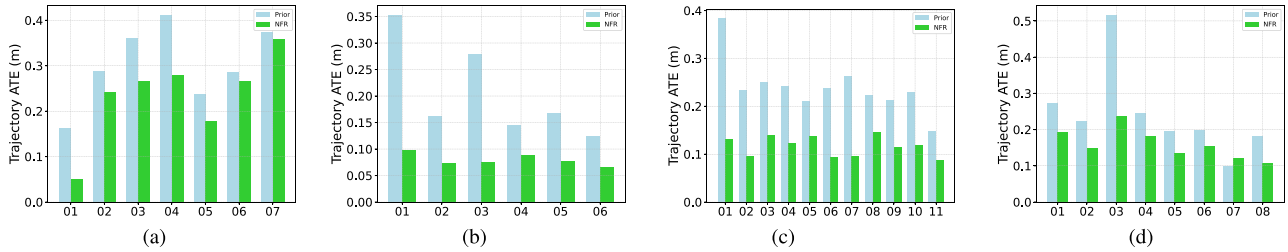


Fig. 16. Quantatitive results of using and without using NFR on mapping accuracy, i.e., the absolute trajectory error. The results indicate that our proposed map-entric NFR performs better than using the prior information matrix, thus demonstrating its effectiveness for the marginalization of long-term mapping. (a) DCC. (b) KAIST. (c) Roundabout. (d) Town.

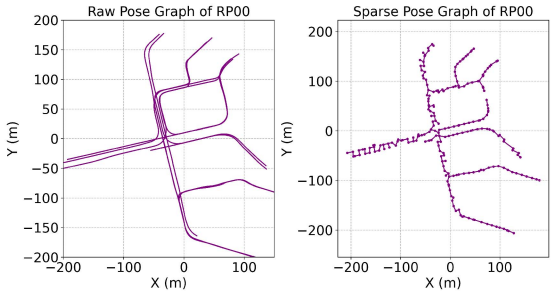


Fig. 17. Reduced sparse pose graph after applying marginalization on HeLiPR. The multisession data results in multiple pose graphs in the same region. The marginalization provides not only a more compact topological structure but also the information matrix for long-term mapping.

NFR unnecessary due to its time costs in solving (18). To verify this, we applied topological reconstruction (keyframe sparsification) in our method while maintaining the prior information described in Section IV-A1, i.e., without solving the NFR problem. Specifically, we define this prior information as $\sqrt{\mathbf{\Lambda}_{r_k}} = \mathbf{I} \cdot \sqrt{N_{r_k}/m}/\sigma$, considering the sum of points in the original observations, as the number of points directly reflects their weights. Fig. 16 indicates that mapping accuracy drops if using priors compared to the full module (NFR enabled). The results demonstrate that NFR plays a vital role in both consistency and scalability for long-term mapping.

The minimum spanning tree generates the sparse topology of keyframe connections. Fig. 17 shows the effectiveness of our keyframe sparsification approach, ensuring comprehensive connectivity among all keyframes while reducing unnecessary information storage. An additional issue in topology reconstruction is whether the system can preserve mapping accuracy when a landmark is retained with only a single observation on the keyframe, i.e., each landmark is connected only to the nearest keyframe pose, as described in Section V-B. Our prior experiments in Fig. 11 show that the system consistently maintained high accuracy following each BA. The optimized results of the LiDAR BA remain unaffected by the precision of the multisession input submaps. These results indicate that the proposed map-centric NFR can provide a reasonable information matrix to ensure the stability of mapping refinement, even when each landmark is connected to one keyframe pose after topology reconstruction.

E. Map Re-Use for Online Localization

SLIM is designed to provide maps that can be reused for online state estimation, i.e., robot localization, a basic need for modern mobile robots. We employ a straightforward LiDAR-based localization algorithm to evaluate whether our generated map can be utilized for robot relocalization. For simplicity, we primarily assess continuous localization capability, while global relocalization can be achieved using the module presented in Section IV-B2. We apply the same feature extraction operations described in Section IV-A1 to the original LiDAR scans to obtain features as measurements. Subsequently, we use the pose from the previous frame $\mathbf{T}_f^w$ as the prior pose for the current frame and perform nearest neighbor data association between the points $\mathbf{p}_l$ and landmarks $(\mathbf{n}_k, \mathbf{p}_k)$ of the current frame, thus obtaining the matching pairs $\mathcal{C}^{\mathcal{L}}$ and $\mathcal{C}^{\mathcal{S}}$. Essentially, localization on the map is to minimize the error between observed points and the prebuilt maps. We adopt a point-to-landmarks residual to achieve online localization, described as follows:

$$\min_{\mathbf{T}_f^w} \sum_{(k,l)\in\mathcal{C}^{\mathcal{L}}} \rho\left(\|(\mathbf{I}-\mathbf{n}_k\mathbf{n}_k^T)\left(\mathbf{T}_f^w\mathbf{p}_l-\mathbf{p}_k\right)\|_{\Lambda_{\mathcal{L}}}^2\right) \sum_{(k,l)\in\mathcal{C}^{\mathcal{S}}} \rho\left(\|\mathbf{n}_k^T\left(\mathbf{T}_f^w\mathbf{p}_l-\mathbf{p}_k\right)\|_{\Lambda_{\mathcal{S}}}^2\right). \tag{33}$$

TABLE III
ONLINE LOCALIZATION ON SLIM-GENERATED MAPS: ABSOLUTE TRAJECTORY ERROR (RMSE, CENTIMETERS)

Region	DP01	DP02	DP03	DP04	DP05	DP06	DP07	KP01
ATE	2.8	8.8	5.8	32.6	6.2	8.5	17.9	7.9
Region	KP02	KP03	KP04	KP05	KP06	RP01	RP02	RP03
ATE	5.9	6.9	8.6	3.3	5.0	8.4	5.5	6.4
Region	RP04	RP05	RP06	RP07	RP08	RP09	RP10	RP11
ATE	9.0	7.1	5.4	5.1	6.9	11.5	9.2	8.6
Region	TP01	TP02	TP03	TP04	TP05	TP06	TP07	TP08
ATE	19.0	11.1	35.8	6.5	9.2	9.2	12.2	14.2

Table III reports quantitative results on the HeLiPR dataset. The localization error is generally at the centimeter level, which is sufficiently precise for localization-oriented applications, such as autonomous driving in urban environments. Note that the error includes the errors of premapping, resulting in larger errors in certain regions. For practical applications, additional information, such as odometry and semantics [8], could improve localization accuracy under a sensor fusion scheme. Moreover, the average latency for solving the scan-to-map optimization using a single thread is approximately 42 ms (CPU: Intel i7-14650HX), which is sufficient for real-time localization applications.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

VII. Discussions

Though the designed SLIM system is versatile for long-term LiDAR mapping, potential limitations exist for practical applications. The two key limitations are listed as follows.

1) A closed-form solution for the NFR requires that the dimension of the recovered residual matches the dimension of the state variables. This requires the Jacobian matrix must be square. In SLAM backends, this condition is not always met. For example, incorporating GPS constraints, pitch-roll constraints, or other constraints may result in mismatched dimensions between residuals and state variables. A crucial factor in our system is that the dimension of lines and planes precisely matches their residual dimension, easily satisfying the condition for a closed-form solution. However, if more information is fused, this condition may not hold.
2) Although an iterative solution for NFR is available [72], the computational cost increases rapidly for extremely large-scale maps. Fortunately, for maps containing between 1.0×10^4 and 1.0×10^5 landmarks, the proposed SLIM can achieve a single-step solution within a few seconds, which is acceptable for iterative methods. If more map landmarks are involved, the SLIM system requires more time to provide the solution.

The SLIM backend remains effective for visual SLAM, as its problem formulation is inherently aligned with the principles of visual BA. For instance, in the context of a crowdsourced semantic mapping task on roads, the line and plane features within the framework can be effortlessly replaced with semantic entities such as poles, lane markings, and traffic signs. By incorporating updated landmark definitions and residual formulations, the back-end solver retains its adaptability to novel problem representations, thereby ensuring robust and precise solutions.

Despite the focus on urban environments, the SLIM system could also be applied to other structured environments, such as forests and indoors. In forests, the ground and trees can be parameterized into planes and lines. Indoor environments are highly structured with elements like walls and columns, which are typical planes and lines. We also want to emphasize that the goal of the SLIM system is not to achieve state-of-the-art performance. This study primarily aims to provide insights into lightweightness and scalability for LiDAR mapping in urban environments, which are the motivations as stated in Section I.

VIII. Conclusion

In this study, we presented SLIM, a lightweight and scalable LiDAR mapping system for urban environments. Our contributions spanned from front-end feature extraction to back-end map optimization and management. Experiments conducted on various datasets with different LiDAR sensor types demonstrated the effectiveness of the SLIM system. The results showed that SLIM can merge, refine, and maintain new maps using only line and plane representations.

Future studies could explore several promising directions. First, SLIM could be deployed in other structured environments similar to urban scenarios. At the front end, learning-based feature extraction could reduce the need for parameter tuning. At the back end, experiments currently rely on manual region divisions, and a more adaptive approach could be a better choice for long-term autonomy.

Appendix

The derivation of map refinement parts is detailed in this section. The first part is the residual and Jacobian matrices with respect to line landmarks and robot poses. As described in Section IV-A, the line landmark is parameterized by two rotation angles (roll and pitch) and two translations

$$\begin{aligned} \mathbf{l}_{\mathcal{L}} &= f(\mathbf{n}, \mathbf{c}) = g(\alpha, \beta, x, y) \\ \mathbf{n} &= \mathbf{R}(\alpha, \beta)\, \mathbf{u}_z \\ \mathbf{q} &= \mathbf{R}(\alpha, \beta)\, (\mathbf{u}_x x + \mathbf{u}_y y). \end{aligned} \tag{34}$$

Theoretically, the degrees of freedom for point-to-line residual is 2 because the constraints along the direction of the line are redundant. Thus, we project the point-to-line vector to the original orthogonal frame based on the definition of line landmark, described as follows:

$$\mathbf{r}_{\mathcal{L}}\left(\mathbf{R}_f^w, \mathbf{p}_f^w, \mathbf{l}_{\mathcal{L}}\right) = \mathbf{R}^T(\alpha, \beta)\left(\mathbf{I}_3 - \mathbf{n}\mathbf{n}^T\right)(\mathbf{p}^w - \mathbf{q}) \tag{35}$$

where $\mathbf{p}^w = \mathbf{R}_f^w \hat{\mathbf{p}} + \mathbf{p}_f^w$ and $\hat{\mathbf{p}}$ is the observation points in local keyframe coordinate. Note that the third dimension of this residual is redundant; we will now derive its equivalent form. From (3) and (34), we can derive the following:

$$\begin{aligned} &\mathbf{R}^T(\alpha, \beta)\left(\mathbf{I}_3 - \mathbf{n}\mathbf{n}^T\right) \\ &\quad = \mathbf{R}^T(\alpha, \beta)\left(\mathbf{I} - \mathbf{R}(\alpha, \beta)\, \mathbf{u}_z \mathbf{u}_z^T \mathbf{R}^T(\alpha, \beta)\right) \\ &\quad = \left(\mathbf{R}^T(\alpha, \beta) - \mathbf{u}_z \mathbf{u}_z^T \mathbf{R}^T(\alpha, \beta)\right) \\ &\quad = \left(\mathbf{I}_3 - \mathbf{u}_z \mathbf{u}_z^T\right) \mathbf{R}^T(\alpha, \beta). \end{aligned} \tag{36}$$

Therefore, the equivalent form of the point-to-line residual can be transformed into the subsequent equation

$$\begin{aligned} \mathbf{r}_{\mathcal{L}}\left(\mathbf{R}_f^w, \mathbf{p}_f^w, \mathbf{l}_{\mathcal{L}}\right) &= \left(\mathbf{I}_3 - \mathbf{u}_z \mathbf{u}_z^T\right) \mathbf{R}^T(\alpha, \beta)(\mathbf{p}^w - \mathbf{q}) \\ &= \left(\mathbf{I} - \mathbf{u}_z \mathbf{u}_z^T\right)\left(\mathbf{R}^T(\alpha, \beta)\, \mathbf{p}^w - (\mathbf{u}_x x + \mathbf{u}_y y)\right) \\ &= \mathbf{R}^T(\alpha, \beta)_{[2\times 3]}\, \mathbf{p}^w - (\mathbf{u}_x x + \mathbf{u}_y y)_{[2\times 1]}. \end{aligned} \tag{37}$$

Then we can obtain the Jacobian matrix

$$\begin{aligned} \mathbf{R}_\alpha &= \begin{bmatrix} 0 & \cos\alpha \sin\beta & -\sin\alpha \sin\beta \\ 0 & -\sin\alpha & -\cos\alpha \end{bmatrix} \\ \mathbf{R}_\beta &= \begin{bmatrix} -\sin\beta & \sin\alpha \cos\beta & \cos\alpha \cos\beta \\ 0 & 0 & 0 \end{bmatrix} \\ \frac{\partial \mathbf{r}_{\mathcal{L}}}{\partial \alpha} &= \mathbf{R}_\alpha \mathbf{p}^w, \qquad \frac{\partial \mathbf{r}_{\mathcal{L}}}{\partial \beta} = \mathbf{R}_\beta \mathbf{p}^w \\ \frac{\partial \mathbf{r}_{\mathcal{L}}}{\partial (x, y)} &= -\mathbf{I}_2, \qquad \frac{\partial \mathbf{r}_{\mathcal{L}}}{\partial \mathbf{p}_f^w} = \mathbf{R}^T(\alpha, \beta)_{[2\times 3]} \\ \frac{\partial \mathbf{r}_{\mathcal{L}}}{\partial \delta\theta_f^w} &= -\mathbf{R}^T(\alpha, \beta)_{[2\times 3]} \left(\mathbf{R}_f^w\right)^T [\hat{\mathbf{p}}]_\times. \end{aligned} \tag{38}$$

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

The second part is the residual and Jacobian matrices with respect to plane landmarks and robot poses. The plane landmark is modeled by two rotation angles (roll and pitch) and one translation, described as follows:

$$\mathbf{l}_{\mathcal{S}} = f(\mathbf{n}, d) = g(\alpha, \beta, d). \tag{39}$$

The point-to-plane residual is defined by the point-to-plane distance as follows:

$$\mathbf{r}_{\mathcal{S}}\left(\mathbf{R}_f^w, \mathbf{p}_f^w, \mathbf{l}_{\mathcal{S}}\right) = \mathbf{n}^T \mathbf{p}^w + d \tag{40}$$

and the Jacobian matrices can be derived as

$$\begin{aligned}
\frac{\partial \mathbf{r}_{\mathcal{S}}}{\partial \alpha} &= (\mathbf{p}^w)^T \begin{bmatrix} 0 \\ \cos\alpha\cos\beta \\ -\sin\alpha\cos\beta \end{bmatrix} \\
\frac{\partial \mathbf{r}_{\mathcal{S}}}{\partial \beta} &= (\mathbf{p}^w)^T \begin{bmatrix} -\cos\beta \\ -\sin\alpha\sin\beta \\ -\cos\alpha\sin\beta \end{bmatrix} \\
\frac{\partial \mathbf{r}_{\mathcal{S}}}{\partial d} &= 1, \qquad \frac{\partial \mathbf{r}_{\mathcal{S}}}{\partial \mathbf{p}_f^w} = \mathbf{n}^T \\
\frac{\partial \mathbf{r}_{\mathcal{S}}}{\partial \delta\theta_f^w} &= -\mathbf{n}^T \left(\mathbf{R}_f^w\right)^T [\hat{\mathbf{p}}]_\times .
\end{aligned} \tag{41}$$

To this end, the Jacobian matrices for map refinement (Section IV-C) are obtained under the scheme of vectorized representations (Section IV-A). The errors of maps are reduced by applying the map refinements, as presented in Table I and Fig. 11, thus validating the effectiveness of the derivations in the appendix.

Supplementary Materials

A video is submitted as a multimedia attachment. A public version can be accessed https://youtu.be/8HQnYMf_BWI online. Code is released at https://github.com/HKUST-Aerial-Robotics/SLIM.

REFERENCES

[1] M. Palieri et al., "Locus: A multi-sensor LiDAR-centric solution for high-precision odometry and 3D mapping in real-time," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 421–428, Apr. 2021.

[2] P. Babin, P. Dandurand, V. Kubelka, P. Giguère, and F. Pomerleau, "Large-scale 3D mapping of subarctic forests," in *Proc. Field Serv. Robot.: Results 12th Int. Conf.*, 2021, pp. 261–275.

[3] M. Buehler, K. Iagnemma, and S. Singh, *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Berlin, Germany: Springer, 2009, vol. 56.

[4] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, Aug. 2022.

[5] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3D mapping framework based on octrees," *Auton. Robots*, vol. 34, pp. 189–206, 2013.

[6] J. Lin et al., "Immesh: An immediate LiDAR localization and meshing framework," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4312–4331, Dec. 2023.

[7] A. Rosinol et al., "Kimera: From slam to spatial perception with 3D dynamic scene graphs," *Int. J. Robot. Res.*, vol. 40, no. 12-14, pp. 1510–1546, 2021.

[8] Z. Yu, Z. Qiao, L. Qiu, H. Yin, and S. Shen, "Multi-session, localization-oriented and lightweight LiDAR mapping using semantic lines and planes," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 7210–7217.

[9] M. Jung, W. Yang, D. Lee, H. Gil, G. Kim, and A. Kim, "HeLiPR: Heterogeneous LiDAR dataset for inter- LiDAR place recognition under spatiotemporal variations," *Int. J. Robot. Res.*, vol. 43, no. 12, pp. 1867–1883, 2023.

[10] J. Zhang and S. Singh, "Loam: LiDAR odometry and mapping in real-time," in *Proc. Robot.: Sci. Syst.*, vol. 2, no. 9, 2014, pp. 1–9.

[11] K. Chen, B. T. Lopez, A.-A. Agha-mohammadi, and A. Mehta, "Direct LiDAR odometry: Fast localization with dense point clouds," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2000–2007, Apr. 2022.

[12] Z. Liu, X. Liu, and F. Zhang, "Efficient and consistent bundle adjustment on LiDAR point clouds," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4366–4386, Dec. 2023.

[13] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li, "Mulls: Versatile LiDAR slam via multi-metric linear least square," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 11633–11640.

[14] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: In defense of point-to-point ICP–simple, accurate, and robust registration if done the right way," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 1029–1036, Feb. 2023.

[15] M. Kaess, "Simultaneous localization and mapping with infinite planes," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2015, pp. 4605–4611.

[16] L. Zhou, "Efficient second-order plane adjustment," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 13113–13121.

[17] G. Ferrer, "Eigen-factors: Plane estimation for multi-frame and time-continuous point cloud alignment," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 1278–1284.

[18] L. Zhao, Y. Wang, and S. Huang, "Occupancy-SLAM: Simultaneously optimizing robot poses and continuous occupancy map," in *Proc. Robot., Sci. Syst*, 2022, pp. 1–13.

[19] Y. Pan, X. Zhong, L. Wiesmann, T. Posewsky, J. Behley, and C. Stachniss, "PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency," *IEEE Trans. Robot.*, vol. 40, pp. 4045–4064, 2024.

[20] H. Yin et al., "A survey on global LiDAR localization: Challenges, advances and open problems," *Int. J. Comput. Vis.*, vol. 132, no. 8, pp. 3139–3171, 2024.

[21] T. Schneider et al., "MAPLAB: An open framework for research in visual-inertial mapping and localization," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1418–1425, Jul. 2018.

[22] Y. Huang, T. Shan, F. Chen, and B. Englot, "Disco-slam: Distributed scan context-enabled multi-robot LiDAR slam with two-stage global-local graph optimization," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 1150–1157, Apr. 2022.

[23] G. Kim, S. Choi, and A. Kim, "Scan context : Structural place recognition robust to rotation and lateral variations in urban environments," *IEEE Trans. Robot.*, vol. 38, no. 3, pp. 1856–1874, Jun. 2022.

[24] P. Yin et al., "Automerge: A framework for map assembling and smoothing in city-scale environments," *IEEE Trans. Robot.*, vol. 39, no. 5, pp. 3686–3704, Oct. 2023.

[25] P. Yin, L. Xu, J. Zhang, and H. Choset, "Fusionvlad: A multi-view deep fusion networks for viewpoint-free 3D place recognition," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2304–2310, Apr. 2021.

[26] K. Vidanapathirana, P. Moghadam, S. Sridharan, and C. Fookes, "Spectral geometric verification: Re-ranking point cloud retrieval for metric localization," *IEEE Robot. Automat. Lett.*, vol. 8, no. 5, pp. 2494–2501, May 2023.

[27] C. E. Denniston et al., "Loop closure prioritization for efficient and scalable multi-robot SLAM," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 9651–9658, Oct. 2022.

[28] Z. Liu et al., "Seqlpd: Sequence matching enhanced loop-closure detection based on large-scale point cloud description for self-driving vehicles," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 1218–1223.

[29] P.-Y. Lajoie, B. Ramtoula, Y. Chang, L. Carlone, and G. Beltrame, "DOOR-SLAM: Distributed, online, and outlier resilient SLAM for robotic teams," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1656–1663, Apr. 2020.

[30] Y. Tian, Y. Chang, F. H. Arias, C. Nieto-Granda, J. P. How, and L. Carlone, "Kimera-multi: Robust, distributed, dense metric-semantic SLAM for multi-robot systems," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2022–2038, Aug. 2022.

[31] K. Ebadi et al., "Lamp: Large-scale autonomous mapping and positioning for exploration of perceptually-degraded subterranean environments," in *Proc. 2020 IEEE Int. Conf. Robot. Autom.*, 2020, pp. 80–86.

[32] J. G. Mangelson, D. Dominic, R. M. Eustice, and R. Vasudevan, "Pairwise consistent measurement set maximization for robust multi-robot map merging," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 2916–2923.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

[33] Y. Latif, C. Cadena, and J. Neira, "Robust loop closing over time for pose graph SLAM," *Int. J. Robot. Res.*, vol. 32, no. 14, pp. 1611–1626, 2013.
[34] Z. Zou et al., "Lta- om: Long-term association lidar–imu odometry and mapping," *J. Field Robot.*, vol. 41, no. 7, pp. 2455–2474, 2024.
[35] Y. Chang et al., "Lamp 2.0: A robust multi-robot SLAM system for operation in challenging large-scale underground environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 9175–9182, Oct. 2022.
[36] H. Yang, P. Antonante, V. Tzoumas, and L. Carlone, "Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1127–1134, Apr. 2020.
[37] X. Liu, Z. Liu, F. Kong, and F. Zhang, "Large-scale LiDAR consistent mapping using hierarchical LiDAR bundle adjustment," *IEEE Robot. Autom. Lett.*, vol. 8, no. 3, pp. 1523–1530, Mar. 2023.
[38] K. Tang et al., "THMA: Tencent HD map ai system for creating HD map annotations," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 13, pp. 15585–15593.
[39] G. Wan et al., "Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 4670–4677.
[40] X. Chen, A. Milioto, E. Palazzolo, P. Giguere, J. Behley, and C. Stachniss, "Suma : Efficient LiDAR-based semantic SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 4530–4537.
[41] T. Kühner and J. Kümmerle, "Large-scale volumetric scene reconstruction using LiDAR," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2020, pp. 6261–6267.
[42] A. Schaefer, D. Büscher, J. Vertens, L. Luft, and W. Burgard, "Long-term urban vehicle localization using pole landmarks extracted from 3-D LiDAR scans," in *Proc. Eur. Conf. Mobile Robots*, 2019, pp. 1–7.
[43] L. Li, M. Yang, L. Weng, and C. Wang, "Robust localization for intelligent vehicles based on pole-like features using the point cloud," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 2, pp. 1095–1108, Apr. 2022.
[44] J.-H. Pauls, B. Schmidt, and C. Stiller, "Automatic mapping of tailored landmark representations for automated driving and map learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 6725–6731.
[45] T. Wen et al., "Roadside HD map object reconstruction using monocular camera," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7722–7729, Jul. 2022.
[46] Z. Liu and F. Zhang, "Balm: Bundle adjustment for LiDAR mapping," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3184–3191, Apr. 2021.
[47] L. Zhou, G. Huang, Y. Mao, J. Yu, S. Wang, and M. Kaess, "PLC-LiSLAM: Lidar SLAM with planes, lines, and cylinders," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7163–7170, Jul. 2022.
[48] C. Xia et al., "Quadric representations for LiDAR odometry, mapping and localization," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 5023–5030, Aug. 2023.
[49] R. Dube et al., "SegMap: Segment-based mapping and localization using data-driven descriptors," *Int. J. Robot. Res.*, vol. 39, no. 2/3, pp. 339–355, 2020.
[50] B. Mildenhall, P.P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
[51] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, "DeepSDF: Learning continuous signed distance functions for shape representation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 165–174.
[52] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, "Occupancy networks: Learning 3D reconstruction in function space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4460–4470.
[53] X. Zhong, Y. Pan, J. Behley, and C. Stachniss, "Shine-mapping: Large-scale 3D mapping using sparse hierarchical implicit neural representations," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 8371–8377.
[54] S. Isaacson, P.-C. Kung, M. Ramanagopal, R. Vasudevan, and K. A. Skinner, "Loner: LiDAR only neural representations for real-time SLAM," *IEEE Robot. Autom. Lett.*, vol. 8, no. 12, pp. 8042–8049, Dec. 2023.
[55] J. Deng et al., "Nerf-loam: Neural implicit representation for large-scale incremental LiDAR odometry and mapping," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 8218–8227.
[56] C. Toft et al., "Long-term visual localization revisited," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 4, pp. 2074–2088, Apr. 2022.
[57] P. Wenzel et al., "4seasons: A cross-season dataset for multi-weather SLAM in autonomous driving," in *Proc. Pattern Recognit.: 42nd DAGM German Conf.*, 2020, pp. 404–417.
[58] H. Yin, Y. Wang, L. Tang, X. Ding, S. Huang, and R. Xiong, "3D LiDAR map compression for efficient localization on resource constrained vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 2, pp. 837–852, Feb. 2021.
[59] M. Labussière, J. Laconte, and F. Pomerleau, "Geometry preserving sampling method based on spectral decomposition for large-scale environments," *Front. Robot. AI*, vol. 7, 2020, Art. no. 572054.
[60] S. Thrun, Y. Liu, D. Koller, A. Y. Ng, Z. Ghahramani, and H. Durrant-Whyte, "Simultaneous localization and mapping with sparse extended information filters," *Int. J. Robot. Res.*, vol. 23, no. 7-8, pp. 693–716, 2004.
[61] J. Vial, H. Durrant-Whyte, and T. Bailey, "Conservative sparsification for efficient and consistent approximate estimation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2011, pp. 886–893.
[62] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.
[63] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intell. Transp. Syst. Mag.*, vol. 2, no. 4, pp. 31–43, Winter 2010.
[64] F. Dellaert, "Factor graphs and GTSAM: A hands-on introduction," Georgia Inst. Technol., Atlanta, GA, USA, Tech. Rep. GT-RIM-CP&R-2012-002, 2012.
[65] V. Ila, J. M. Porta, and J. Andrade-Cetto, "Information-based compact pose SLAM," *IEEE Trans. Robot.*, vol. 26, no. 1, pp. 78–93, Feb. 2010.
[66] H. Kretzschmar and C. Stachniss, "Information-theoretic compression of pose graphs for laser-based SLAM," *Int. J. Robot. Res.*, vol. 31, no. 11, pp. 1219–1230, 2012.
[67] N. Carlevaris-Bianco, M. Kaess, and R. M. Eustice, "Generic node removal for factor-graph SLAM," *IEEE Trans. Robot.*, vol. 30, no. 6, pp. 1371–1385, Dec. 2014.
[68] K. J. Doherty, D. M. Rosen, and J. J. Leonard, "Spectral measurement sparsification for pose-graph SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 1–8.
[69] K. Khosoussi, M. Giamou, G. S. Sukhatme, S. Huang, G. Dissanayake, and J. P. How, "Reliable graphs for SLAM," *Int. J. Robot. Res.*, vol. 38, no. 2/3, pp. 260–298, 2019.
[70] G. Kurz, M. Holoch, and P. Biber, "Geometry-based graph pruning for lifelong SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021, pp. 3313–3320.
[71] M. Mazuran, G. D. Tipaldi, L. Spinello, and W. Burgard, "Nonlinear graph sparsification for SLAM," in *Proc. Robot.: Sci. Syst.*, 2014, pp. 1–8.
[72] M. Mazuran, W. Burgard, and G. D. Tipaldi, "Nonlinear factor recovery for long-term SLAM," *Int. J. Robot. Res.*, vol. 35, no. 1–3, pp. 50–72, 2016.
[73] J. Vallvé, J. Solà, and J. Andrade-Cetto, "Graph SLAM sparsification with populated topologies using factor descent optimization," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 1322–1329, Apr. 2018.
[74] B. Jiang and S. Shen, "A two-step nonlinear factor sparsification for scalable long-term SLAM backend," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 10253–10259.
[75] L. Paull, G. Huang, M. Seto, and J. J. Leonard, "Communication-constrained multi-AUV cooperative SLAM," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2015, pp. 509–516.
[76] V. Usenko, N. Demmel, D. Schubert, J. Stückler, and D. Cremers, "Visual-inertial mapping with non-linear factor recovery," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 422–429, Apr. 2020.
[77] M. Oh et al., "TRAVEL: Traversable ground and above-ground object segmentation using graph representation of 3D LiDAR scans," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7255–7262, Jul. 2022.
[78] Z. Qiao, Z. Yu, B. Jiang, H. Yin, and S. Shen, "G3reg: Pyramid graph-based global registration using Gaussian ellipsoid model," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 3416–3432, 2025.
[79] P. C. Lusk, D. Parikh, and J. P. How, "Graffmatch: Global matching of 3D lines and planes for wide baseline LiDAR registration," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 632–639, Feb. 2023.
[80] R. A. Rossi, D. F. Gleich, and A. H. Gebremedhin, "Parallel maximum clique algorithms with applications to network analysis," *SIAM J. Sci. Comput.*, vol. 37, no. 5, pp. C589–C616, 2015.
[81] P. J. Besl and N. D. McKay, "Method for registration of 3-D shapes," in *Proc. SPIE*, vol. 1611, pp. 586–606, 1992.
[82] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment—a modern synthesis," in *Proc. Vis. Algorithms: Theory Practice: Int. Workshop Vis. Algorithms*, 2000, pp. 298–372.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.

[83] N. Carlevaris-Bianco and R. M. Eustice, "Generic factor-based node marginalization and edge sparsification for pose-graph SLAM," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2013, pp. 5748–5755.
[84] Y. Fan, T. Zhao, and G. Wang, "Schurvins: Schur complement-based lightweight visual inertial navigation system," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 17964–17973.
[85] J. Hsiung, M. Hsiao, E. Westman, R. Valencia, and M. Kaess, "Information sparsification in visual-inertial odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 1146–1153.
[86] M. A. Woodbury, *Inverting Modified Matrices*. Department of Statistics, Princeton, NJ, USA: Princeton Univ., 1950.
[87] G. Guennebaud et al., "Eigen v3," 2010. [Online]. Available: http://eigen.tuxfamily.org
[88] P. NVIDIA Vingelmann and F. H. Fitzek, "Cuda release: 10.2.89," 2020. [Online]. Available: https://developer.nvidia.com/cuda-toolkit
[89] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The Kitti vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3354–3361.
[90] N. Carlevaris-Bianco, A. K. Ushani, and R. M. Eustice, "University of Michigan north campus long-term vision and LiDAR dataset," *Int. J. Robot. Res.*, vol. 35, no. 9, pp. 1023–1035, 2016.
[91] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Voxelized gicp for fast and accurate 3D point cloud registration," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 11054–11059.
[92] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "CT-ICP: Real-time elastic LiDAR odometry with loop closure," in *Proc. Int. Conf. Robot. Autom.*, 2022, pp. 5580–5586.
[93] L. Zhou, S. Wang, and M. Kaess, "π-LSAM: LiDAR smoothing and mapping with planes," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 5751–5757.
[94] H. Huang et al., "On bundle adjustment for multiview point cloud registration," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8269–8276, Oct. 2021.
[95] J. Yin, A. Li, T. Li, W. Yu, and D. Zou, "M2DGR: A multi-sensor and multi-scenario SLAM dataset for ground robots," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2266–2273, Apr. 2022.
[96] J. Behley and C. Stachniss, "Efficient surfel-based SLAM using 3D laser range data in urban environments," in *Proc. Robot.: Sci. Syst.*, vol. 2018, 2018.
[97] I. Vizzo, X. Chen, N. Chebrolu, J. Behley, and C. Stachniss, "Poisson surface reconstruction for LiDAR odometry and mapping," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 5624–5630.
[98] I. Vizzo, T. Guadagnino, J. Behley, and C. Stachniss, "Vdbfusion: Flexible and efficient TSDF integration of range sensor data," *Sensors*, vol. 22, no. 3, 2022, Art. no. 1296.
[99] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-LIO: Lightweight tightly coupled LiDAR-inertial odometry using parallel sparse incremental voxels," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 4861–4868, Apr. 2022.



Zehuan Yu received the B.Eng. degree in mechanical engineering from the Harbin Institute of Technology, Harbin, China, in 2019, and the M.S. degree in control science and engineering from the Harbin Institute of Technology (Shenzhen), Shenzhen, China, in 2022. He is currently working toward the Ph.D. degree in electronic and computer engineering with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong.

His research interests include robot perception, SLAM, and computer vision.



Zhijian Qiao received the B.Eng. degree in automation from Northeastern University, Shenyang, China, in 2019, and the M.S. degree in control engineering from Shanghai Jiao Tong University, Shanghai, China, in 2022. He is currently working toward the Ph.D. degree in electronic and computer engineering with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong.

His research interests include the areas of robotics and autonomous driving, with focus on state estimation, localization and mapping, and computer vision.



Wenyi Liu received the B.Eng. degree in communication engineering from the Harbin Institute of Technology (Shenzhen), Shenzhen, China, in 2022. He is currently working toward the Ph.D. degree in mechanical engineering with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong.

His research interests include robot, motion planning, and control.



Huan Yin (Member, IEEE) received the B.Eng. degree in electronic and instrumentation, and the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2016 and 2021, respectively.

He was a Visiting Scholar with the University of Technology Sydney, Australia, in 2018. After graduation, he was a Research Fellow with the Advanced Robotics Centre, National University of Singapore. He then joined the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, as a Research Assistant Professor. His research interests include robot perception, localization and mapping, and autonomous navigation.



Shaojie Shen received the B.Eng. degree in electronic engineering from the Hong Kong University of Science and Technology, Hong Kong, in 2009, and the M.S. degree in robotics and the Ph.D. degree in electrical and systems engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2011 and 2014, respectively.

In September 2014, he joined the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, as an Assistant Professor, and was promoted to Associate Professor in July 2020. His research interests include robotics and unmanned aerial vehicles, with a focus on state estimation, sensor fusion, computer vision, localization and mapping, and autonomous navigation in complex environments.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:38 UTC from IEEE Xplore. Restrictions apply.